

Formål med pentesten:

Dette oppdraget har som formål å teste ens ferdigheter på penetrasjon etter å ha jobbet med boken “Kali Linux 2018: Assuring Security by Penetration Testing”. Boken tar for seg alle mulige svakheter som finnes i forskjellige VM og hvordan en utnytter disse sårbarhetene. VM-målet har navnet “Eksamen_2023” og grunnen til at en velger dette er fordi det er ikke et like kjent VM, i motsetning til DVWA for eksempel, noe som gjør den unik, men også mer utfordrende da en ikke kan google etter en guide på for utnyttelsen av sårbarhet på en service, port eller javascript kode.

En sekundær grunn til å utføre pentesten nå i 2026 er fordi en har tid til å fokusere kun på pentesten. Da en gjorde pentesten første gangen i 2023, så var det i eksamens kontekst og inkludert med at pentesten hadde samme tidsramme, så måtte en også jobbe med to andre fag samtidig, noe som reduserte tida en hadde for pentesten. Dette gjorde at potensielle svakheter som en kunne ha utnyttet og dokumentert da, ble ikke utført pga av den stramme timeplanen

Dette pentestoppdraget varer fra 30.03/2026 til 30.05/2026 og alle funn og fremgangsmåter for hvert funn vil bli dokumentert med skjermbilder og tekst, samtidig som sårbarhetene blir klassifisert i en skala fra kritisk til lavt nivå. Social engineering teknikker er out of scope da VM-en ikke har menneskelige faktorer bak seg og med tanke på loven.

Executive summary:

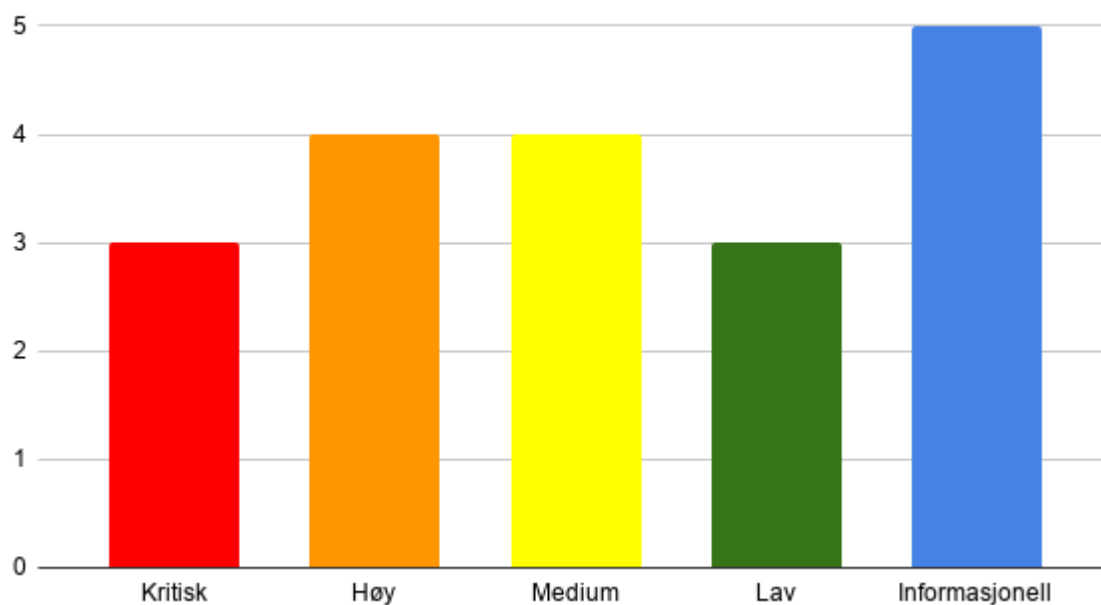
Det ble utført en sikkerhetsrevisjon av webapplikasjonen til Borislockpicks i perioden 30.03/2026 - 30.05/2026 hvor formålet var å avdekke og dokumentere eksisterende sårbarheter på siden. Testen var i form av en “white-box” test, der en hadde tilgang til kildekoden.

Nettsiden har et moderat sikkerhetsnivå, da det ikke var identifisert angrepsvektorer som utgjorde full tilgang til root. Siden har derimot noen få kritiske sårbarheter og en del høye sårbarheter som svekker sikkerheten til siden. Disse er da: SQL-injection, backend-sider sårbare mot ordlisteangrep, IDOR mot brukerid og en miskonfigurering av FTP. Disse svakhetene til sammen fremviser sensitive kundeinformasjon og infrastruktur for bedriften, der risikoen gjelder alt fra stjålet kunde identitet til nedgang av siden gjennom bruk av SQL-injection eller andre payloads.

Bedriften som hoster siden bør prioritere å sanitere input felter, implementere autentifisering mekanismer som MFA i kombinasjon med passord og brukernavn og unngå å legge sensitive informasjon som passord og brukernavn i plaintext.

Oppsummering av sårbarhetsfunn:

Sårbarhetsfordeling



Kategorisering av sårbarhetsfunn:

Hvert funn er kategorisert med en fargekode, CVSS score, tittel og en risikovurdering av sårbarhetene.

Kategoriene er som følger: Kritisk risiko (rød), høy risiko (oransje), medium risiko (gult), lav risiko (grønn) og informasjonell (blått):

Sårbarhetsfarge	Rød
Kritisk risiko	Sårbarheter som må behandles umiddelbart, da disse utgjør en stor risiko for bedriftens nettverk, systemer og data. Slike sårbarheter oftest krever ikke dypt kunnskap for å utnytte

Sårbarhetsfarge	Oransje
Høy risiko	Sårbarheter som kan utgjøre stor skade for bedriftens verdier da disse utgjør en stor risiko for bedriftens nettverk, systemer og data. Slike sårbarheter krever oftest mer kunnskap for å utnytte

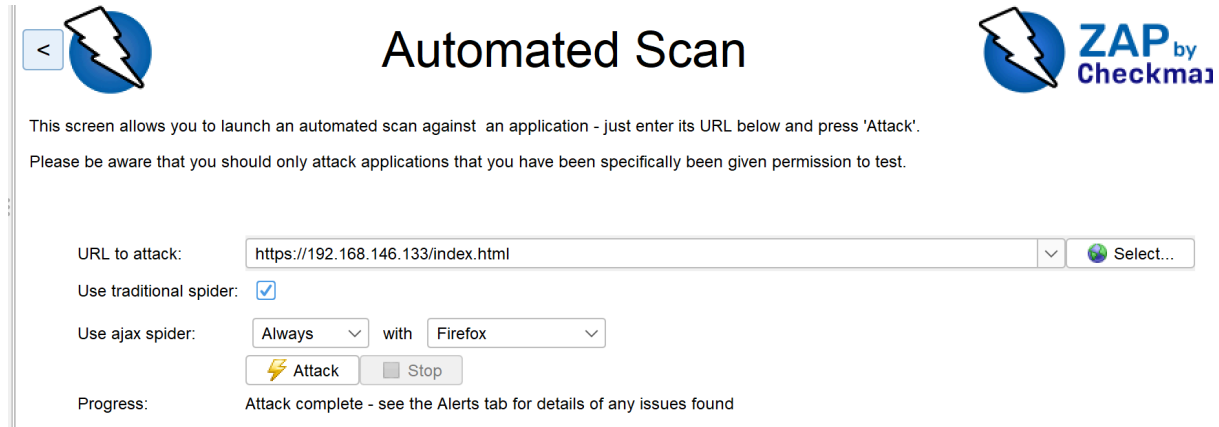
Sårbarhetsfarge	Gul
Medium risiko	Sårbarheter som ikke nødvendigvis gir katastrofale tap på bedriftens nettverk, systemer og data. Bør fiskes så snart man har tid Slike sårbarheter er vanskeligere å utnytte, oftest utført gjennom flere steg

Sårbarhetsfarge	Grønn
Lav risiko	Sårbarheter som oftest skader ikke-kritiske infrastruktur til bedrifter. Bør fiskes når andre mer sårbare svakheter er håndtert Slike sårbarheter krever mye arbeid og riktige forhold for å utnyttes

Informasjonell	Blått
Informasjon	Funn som ikke direkte forårsaker en sårbarhet i infrastrukturen til bedriftene. Disse inneholder informasjon som kan hjelpe angripere med reconnaissance arbeidet

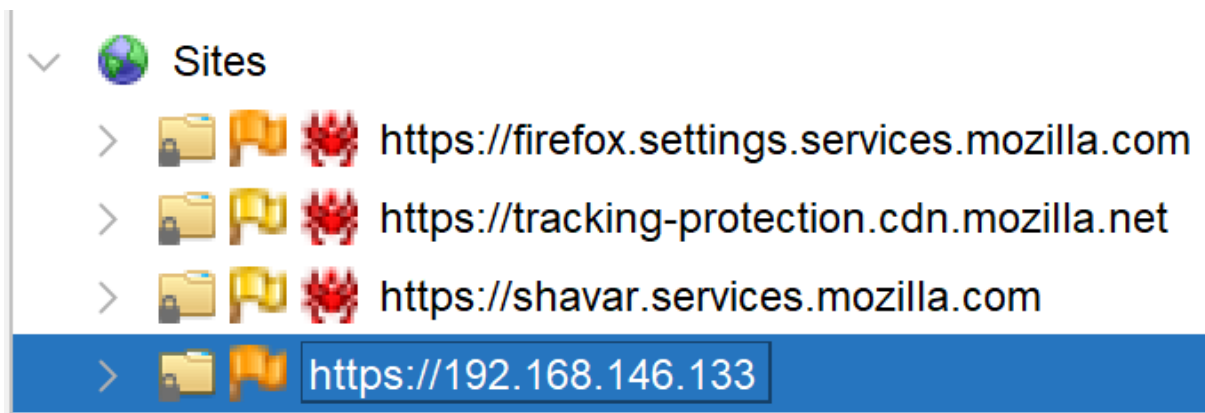
Fremgangsmåte for Reconissance:

Startet først med å utføre en OWASP ZAP scan mot VM-en, der en også inkluderte bruken av normal “Spider” og “Ajax Spider”:



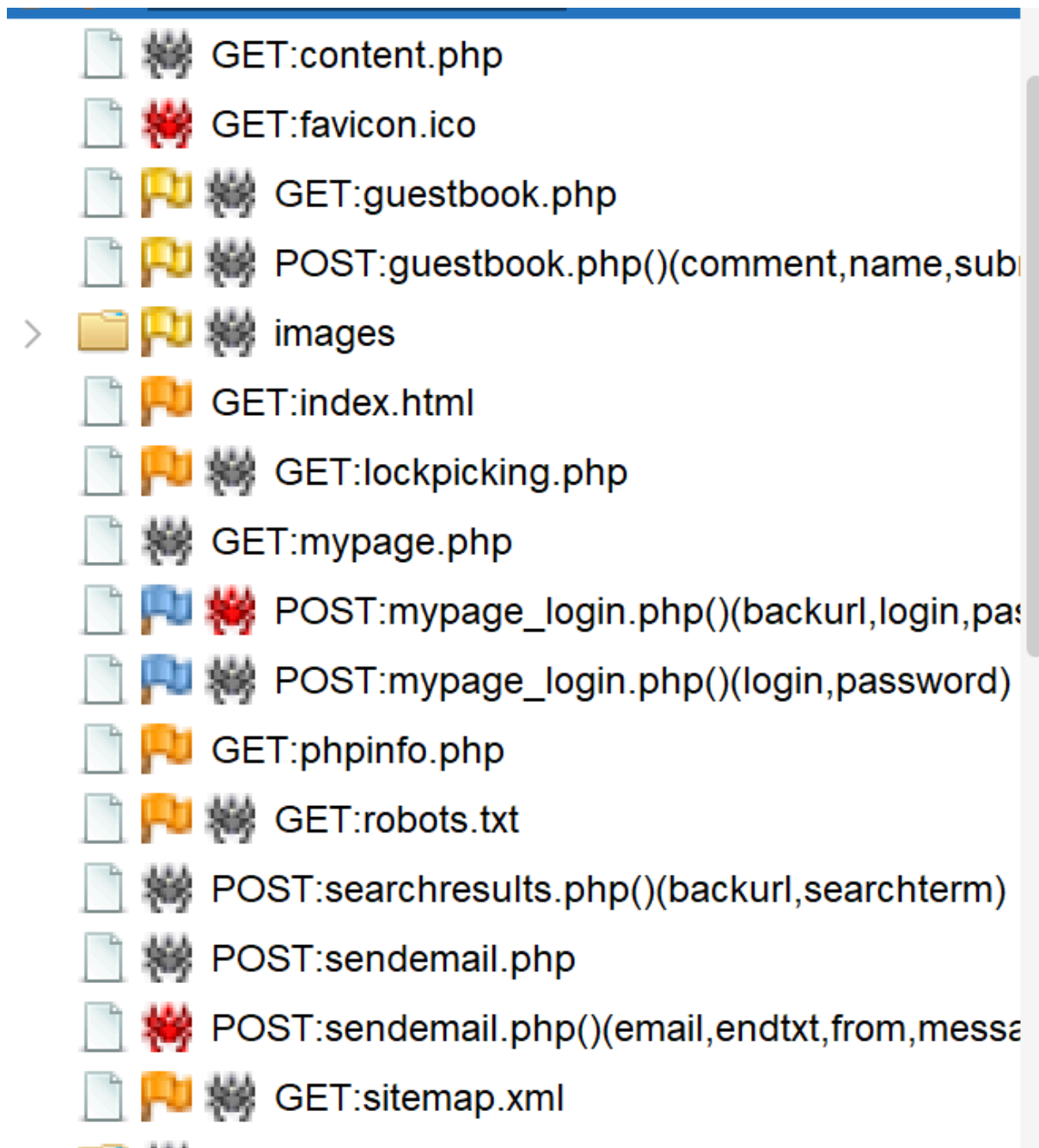
Figurbilde 1: Automatisk scan mot VM-ens IP-adresse, der “Use Traditional spider” er sjekket og valget om å bruke “ajax spider” mot “Firefox” er satt som “Always”.

Får da opp alle sider som OWASP ZAP har scannet gjennom spiderne, der det mest interessante er siden som er i bunnen:



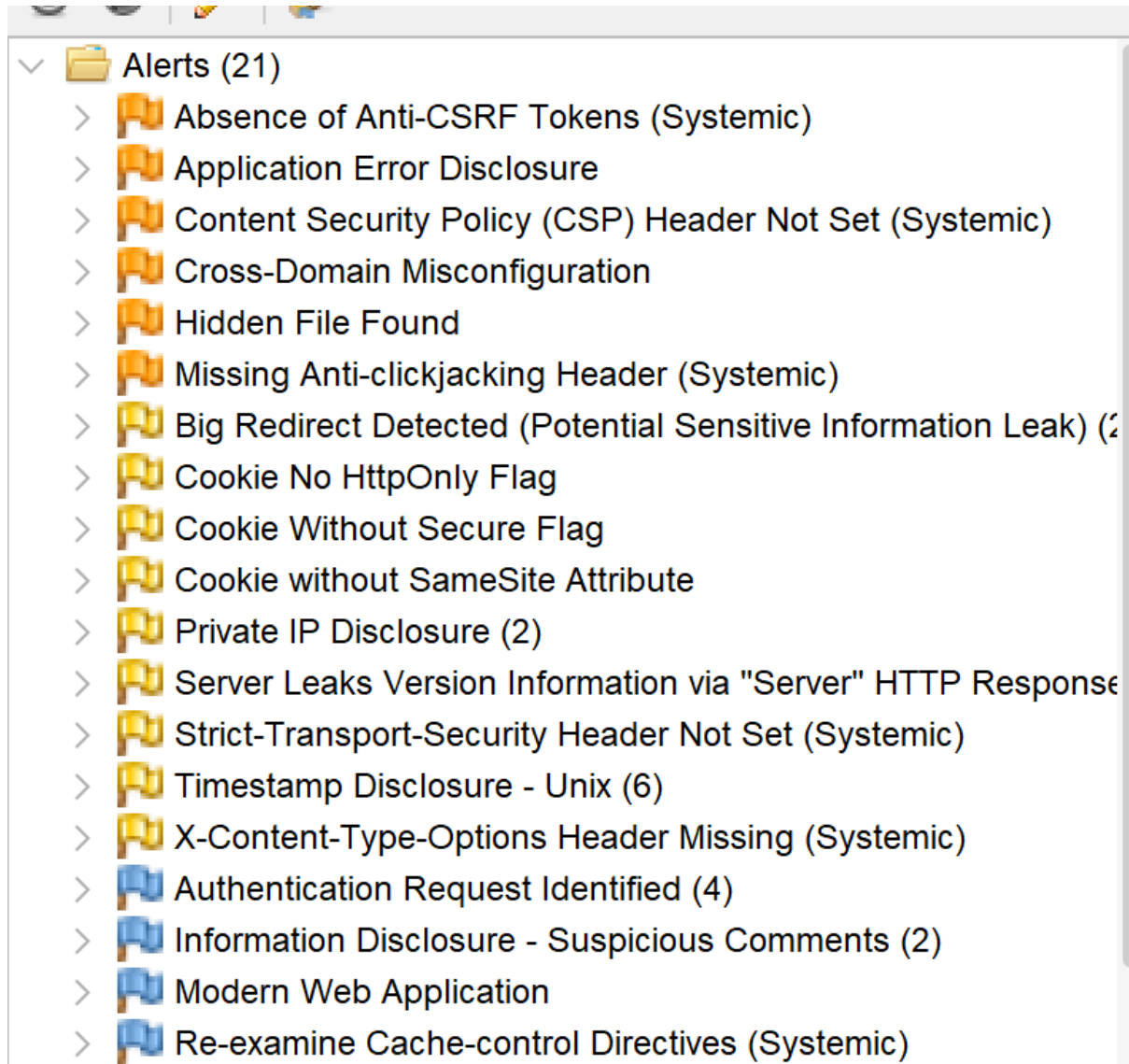
Figurbilde 2: Liste av alle “sites” som OWASP ZAP har besøkt. Browseren som ble brukt er FireFox.

Skjermbildet under viser alle sidene som OWASP ZAP har “spiderert” gjennom, inkludert interessante sider som “phpinfo.php”



Figurbilde 3: Interessante filstiler som OWASP ZAP har funnet i nettsiden

Alle alerts som OWASP ZAP har funnet frem. Noe av disse er lite brukbart, mens andre som “Hidden File Found” alerten kan lede til en sårbarhet:



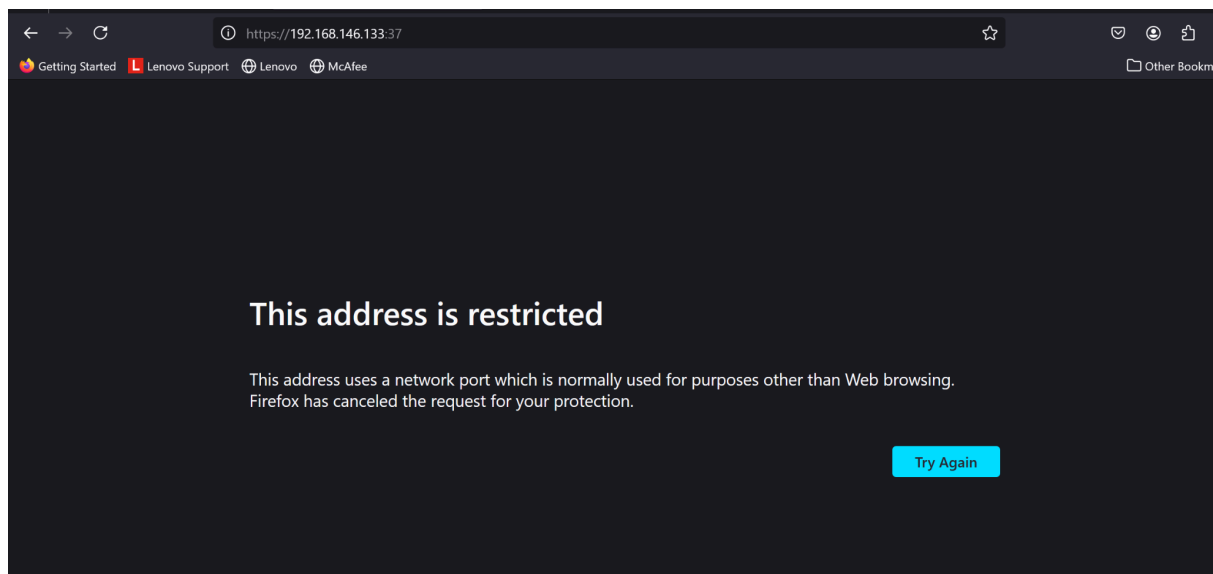
Figurbilde 4: Liste av alle alerts, inkludert “Hidden File Found” alerten

Brukte Kali Linux for å utføre en enkel NMAP scan mot VM-en, der en fikk en oversikt over alle åpne porter:

```
$ nmap 192.168.146.133
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-30 08:22 EDT
Nmap scan report for 192.168.146.133
Host is up (0.00076s latency).
Not shown: 988 closed tcp ports (reset)
PORT      STATE SERVICE
9/tcp     open  discard
13/tcp    open  daytime
21/tcp    open  ftp
22/tcp    open  ssh
37/tcp    open  time
53/tcp    open  domain
79/tcp    open  finger
80/tcp    open  http
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
9999/tcp  open  abyss
MAC Address: 00:0C:29:08:07:C1 (VMware)
```

Figurbilde 5: Enkel nmap scan av VM-en der den lister opp alle åpne porter og tilkoblede servicer

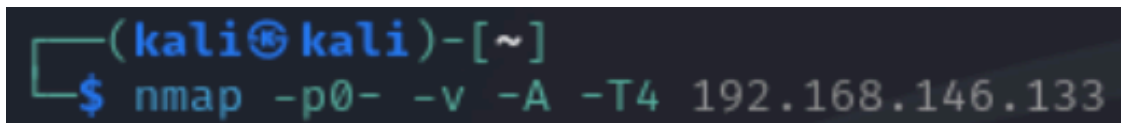
Testet ut alle portene fra scannen og porter 9, 13, 21, 22, 37, 53, 79 og 139 responderer med meldingen “This Adress is restricted”:



Figurbilde 6: Meldingen “This Adress is restricted” når en skriver IP-adressen og portnummer 37

Interessant nok, så skjer det ingen endring når en erstatter 37 med 443, da det er porten som nettsiden kjører på (se figurbilde 64).

Like etterpå modifiserte en nmap scannen slik at den kunne gi mer info om servicene koblet til hver port. Parameteren “-p0-” scanner alle porter fra 0-65535, -v betyr “verbose” og gir mer detaljert info om funnene, “-A” gjør at scannen også detekterer operativsystemet, versjonsnummeret til de ulike servicene, gjennomføring av scriptscanning og implementering av traceroute funksjonen. “-T4” er hastigheten for scanningen, tilsvarende “Aggressive” :



```
(kali@kali)-[~]  
$ nmap -p0- -v -A -T4 192.168.146.133
```

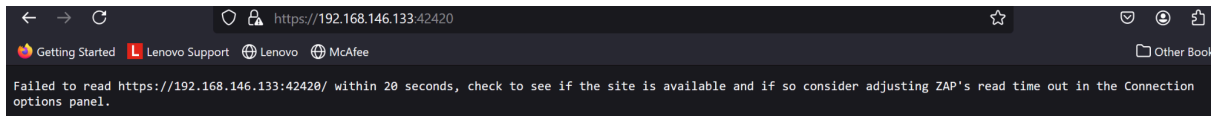
Figurbilde 7: Utvidet nmap scan.



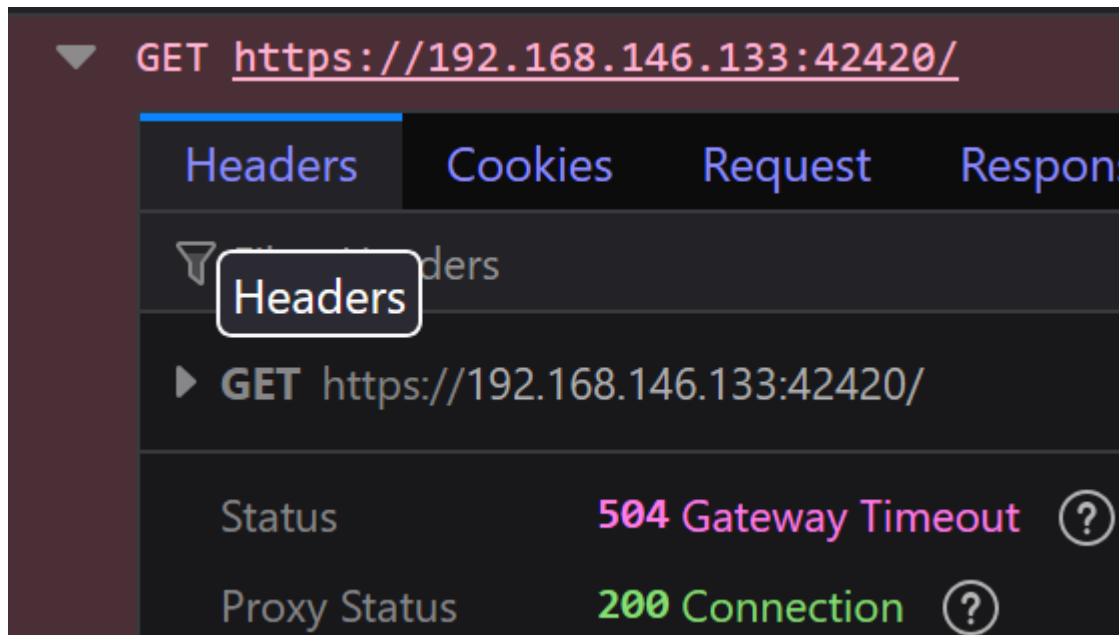
```
9/tcp    open  discard?  
13/tcp   open  daytime  
21/tcp   open  ftp?  
22/tcp   open  ssh      OpenSSH 7.9p1 Debian 10+deb10u3 (protocol 2.0)  
| ssh-hostkey:  
| 2048 7e:20:d9:24:45:11:ee:34:77:03:d7:f4:25:c5:95:bb (RSA)  
| 256 8f:d1:99:c9:6c:ba:34:de:f2:6a:dc:2b:91:60:61:b5 (ECDSA)  
| 256 fc:ff:40:0e:72:bd:9c:a0:69:dd:69:4a:7e:e3:44:d4 (ED25519)  
37/tcp   open  time      (64 bits)  
|_ rfc868-time: 2026-03-30T12:26:14  
53/tcp   open  domain    ISC BIND 9.11.5-P4-5.1+deb10u9 (Debian Linux)  
| dns-nsid:  
|_ bind.version: 9.11.5-P4-5.1+deb10u9-Debian  
79/tcp   open  finger?  
80/tcp   open  http      Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)  
|_ http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1  
|_ http-methods:  
|_ Supported Methods: GET HEAD POST OPTIONS  
|_ http-title: Did not follow redirect to https://192.168.146.133/  
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)  
443/tcp  open  ssl/http  Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)  
|_ http-title: Boris' Lockpick SUPERSALE  
|_ http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1  
|_ tls-alpn:  
|_ h2  
|_ http/1.1  
|_ ssl-cert: Subject: commonName=borislockpick.local/organizationName=Boris Lockpicks/stateOrProvinceName=OSLO/countryName=NO  
|_ Subject Alternative Name: DNS:borislockpick.local  
|_ Issuer: commonName=borislockpick.local/organizationName=Boris Lockpicks/stateOrProvinceName=OSLO/countryName=NO  
|_ Public Key type: rsa  
|_ Public Key bits: 1024  
|_ Signature Algorithm: sha256WithRSAEncryption  
|_ Not valid before: 1979-01-01T00:00:00  
|_ Not valid after: 2029-12-31T23:59:59  
|_ MD5: 65f7:5b8f:8b87:b6e9:cbbf:976c:7a04:09e0  
|_ SHA-1: 20c2:81ab:2b37:f856:fac9:a69f:e8cc:28b8:1fc5:9b9f  
|_ ssl-date: TLS randomness does not represent time  
|_ http-methods:  
|_ Supported Methods: OPTIONS HEAD GET  
|_ tls-nextprotoneg:  
|_ h2  
|_ http/1.1  
445/tcp  open  netbios-ssn Samba smbd 4.9.5-Debian (workgroup: WORKGROUP)  
9999/tcp open  http      Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)  
|_ http-title: Error 401  
|_ http-auth:  
|_ HTTP/1.1 401 Unauthorized\x00  
|_ Basic Realm=Abyss Web Server Console  
|_ http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1  
|_ http-favicon: Unknown favicon MD5: F21771C79C81799C8C3109C46FE22B4C  
42420/tcp open  http      lighttpd 1.4.53  
|_ http-methods:  
|_ Supported Methods: OPTIONS GET HEAD POST  
|_ http-title: Medium vulnerability: Port 42420  
|_ http-server-header: lighttpd/1.4.53
```

Figurbilde 8: Liste av servicer kjørerne på de åpne portene, inkludert detaljer om servicenes versjonsnummer.

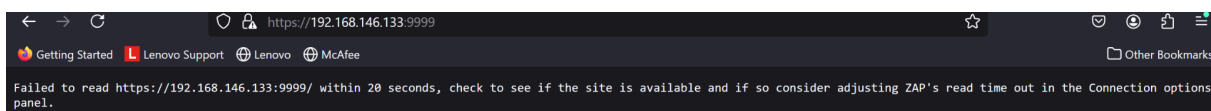
De mer interessante portene er port 9999 og port 42420. Da en prøvde å legge begge portene inn så får en opp dette:



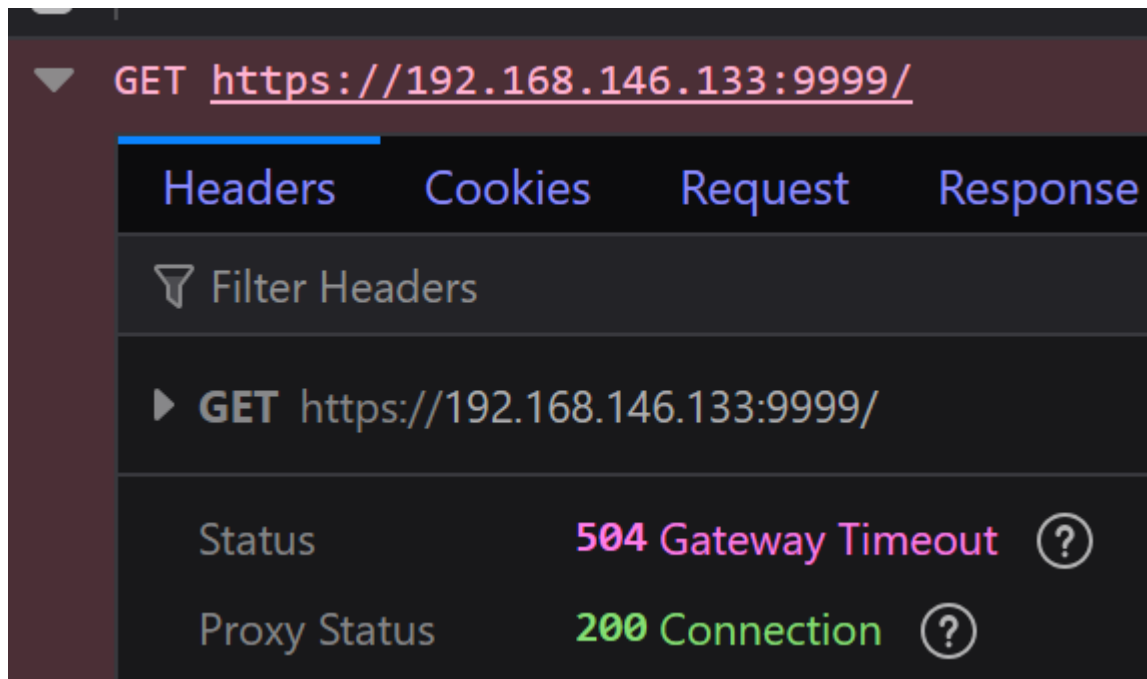
Figurbilde 9: Response etter å lagt til 42420 bak IP-adressen



Figurbilde 10: Konsollen i Firefox rapporter at connection ble etablert, men at responsen fra siden tok lang tid



Figurbilde 11: Response etter å lagt til 9999 bak IP-adressen



Figurbilde 12: Konsollen i Firefox rapporterer at connection ble etablert, men at responsen fra siden tok lang tid

Det figurbilde 9-12 viser er at Firefox fikk en respons fra servicene som kjører på portene, men at backend-applikasjoner ikke klarte å svare i tide. Etter å ha søkt litt på nett, så var det naturlig løsningen å fjerne vekk “https://” i URLen og teste på nytt. Dette gir resultater som blir demonstrert i detalj videre i rapporten, i kapittelet “Sårbarhetsfunn”. Noen av sårbarhetene ble gjort slik som vist i dette kapittelet mens andre ble funnet gjennom manuell testing og undersøkelse.

Sårbarhetsfunn:***Kritisk risiko:***

Sårbarhet 1	Web server sårbarhet
Kritisk risiko	Backend info sårbare for wordlist angrep

En har klart å finne backendsiden til nettsiden ved bruk av verktøyet Dirb. Dirb er en web content scanner som brukes for å finne skjulte filer og filstier på en nettside, hvor den blant annet fant “backend” filstien ved å bruteforce nettsiden mot den valgte tekstfilen “common.txt”:

```
(kali㉿kali)-[~]
$ dirb https://192.168.146.133/

_____
DIRB v2.22
By The Dark Raver
_____

START_TIME: Wed Apr  1 17:12:19 2026
URL_BASE: https://192.168.146.133/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

_____

GENERATED WORDS: 4612

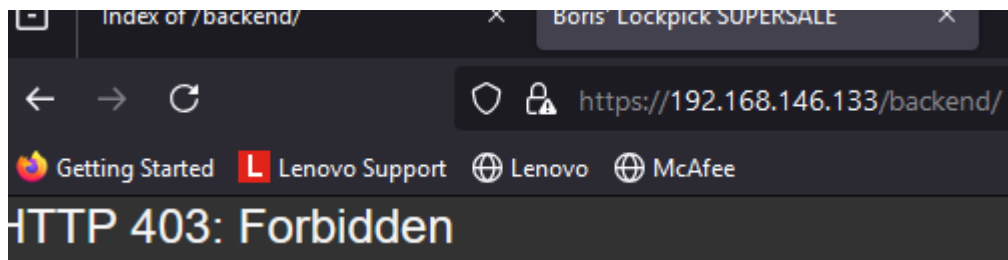
— Scanning URL: https://192.168.146.133/ —
+ https://192.168.146.133/backend (CODE:200|SIZE:267)
+ https://192.168.146.133/docs (CODE:301|SIZE:338)
+ https://192.168.146.133/images (CODE:301|SIZE:342)
+ https://192.168.146.133/index.html (CODE:200|SIZE:1968)
+ https://192.168.146.133/media (CODE:301|SIZE:340)
+ https://192.168.146.133/phpinfo.php (CODE:200|SIZE:90252)
+ https://192.168.146.133/store (CODE:301|SIZE:340)

_____
metasploit

END_TIME: Wed Apr  1 17:13:27 2026
DOWNLOADED: 4612 - FOUND: 7
```

Figurbilde 13: Alle skjulte filstier som dirb har funnet på nettsiden, inkludert “/backend”

En prøvde da å skrive “/backend” etter IP-adressen, men fikk error meldingen “HTTP 403: Forbidden” som resultat. Dette er tydelig tegn på at backend siden eksisterer, men at en ikke har tillatelse til å akkessere den, som vist i figurbilde 14:



Figurbilde 14: Error meldingen “HTTP 403: Forbidden” kommer frem når en skriver inn “/backend” etter IP-adressen

Da ble tanken å legge til en ekstra ‘/’ foran ‘backend’ og da fikk en tilgang til backend siden:

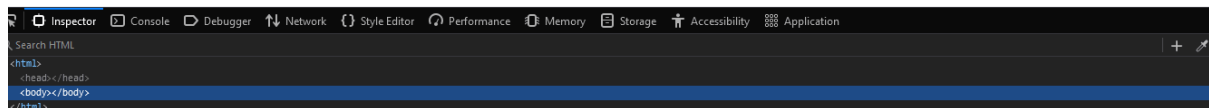
Index of /backend/

Name	Size	Date	MIME Type
../	-	Oct 06, 2023 06:24:10	Directory
comments.data	1.11 MB	Apr 01, 2026 09:56:49	application/octet-stream
security.php	0.18 KB	Oct 06, 2023 06:06:30	application/octet-stream
sendemail.c	0.60 KB	Feb 24, 2023 17:24:12	text/x-c
sendemail_linux	16.27 KB	Mar 01, 2023 18:51:10	application/octet-stream
sendemail_osx_arm	32.65 KB	Mar 04, 2023 17:39:40	application/octet-stream
sendemail_win.exe	46.00 KB	Feb 24, 2023 17:12:28	application/x-msdownload
settings.conf	0.24 KB	Oct 07, 2023 15:46:50	text/plain

Powered by *Abyss Web Server X1*
Copyright © [Aprelium](#) - 2001-2023

Figurbilde 15: Filene som ligger i backend siden av nettsiden

En skulle tro at “security.php” inneholdt noe interessant som policier eller kredensialer for andre servicer, men den viser bare en tom HTML side:



Figurbildet 16: Innholdet i “security.php”

“settings.conf” derimot viser database kredensialene til databasen “borislockpicks”, inkludert root directoriet i plaintext:

```
<?php

$rootdir = "/home/boris/apps/abyssws/htdocs";
$siteroot = "http://www.borislockpick.local";

// Database server:
$dbserver = "localhost";
$dbname = "borislockpicks";
$dbusername = "admin";
$dbpassword = "correcthorse";

?>
```

Figurbildet 17: Database kredensialer for database serveren, inkludert root directory i “settings.conf”

Dette er allerede et kritisk problem for webserveren da kredensialene til Borislockpicks databasen er lett tilgjengelig for angripere å lese fra, ettersom filen er lagt til i backenden uten noen form for kryptering eller autentiseringsmekanisme. Konsekvensene er at angriperen kan få tilgang til database schemaen “borislockpicks” (se figur bilde 43) og utføre uautoriserte handlinger, som å stjele kredensialer til brukerne, modifisere data til brukere, ødelegge dataen

i databasen til nettsiden eller stjele business hemmeligheter hvis de også er lagt til i backenden. En prøvde også å få tilgang til databasen, men det krever at man starter opp mysql på VM-en noe som ikke er helt innenfor scope. Anbefaling er å kryptere og salte data, spesielt sensitive data som database kredensialer, gjennom hashing algoritmer som sha256 eller sha512.

Inkludert med det, så bør “/backend” siden ha en form for autentiseringmekanisme som passord og brukernavn for å forhindre at angriperne kan bare skrive “/backend” etter URL-en og få tilgang. Passorder bør helst ikke være lett å brute force, der det samme ble utført mot Abyss Web Server Consolen for sårbarhet 9, der en bør helst legge til random pauser for hver test, gi randome HTTP koder for mislyktes brute force og bruk av CAPTCHAS, da maskiner ikke klarer å håndtere disse. Dessverre er ikke dette en idiotsikker strategi, med tanke på at AI kan i dag håndtere captchas og den beste løsningen blir å legge til så mange hindringer som mulig slik at angriperen mister motivasjonen til å fortsette med angrepet.

Sårbarhet 2	Miskonfigurering sårbarhet
Kritisk risiko	Port 21 eksponerer sensorveiledning

I nmap-scannen fra figurbilde 8, så stod det at VM-en hadde port 21 åpen. Dette er vanligvis ikke en port man holder åpen, i hvert fall ikke på et VM, med mindre man ønsker å sende filer til og fra VM-en. Da var spørsmålet om en kunne aksessere FTP protokollen som kjører på VM-en:

```
(kali@kali)-[~]  
$ ftp 192.168.146.133  
Connected to 192.168.146.133.  
220 ProFTPD Server (BorisLockpick) [192.168.146.133]  
Name (192.168.146.133:kali): anonymous  
331 Anonymous login ok, send your complete email address as your password  
Password:  
230-Welcome, archive user anonymous@192.168.146.129 !  
230-  
230-The local time is: Wed Apr 01 09:09:39 2026 28.64 seconds  
230-  
230-This is an experimental FTP server. If you have any unusual problems,  
230-please report them via e-mail to <root@osboxes>.  
230-  
230 Anonymous access granted, restrictions apply  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp>
```

Figurbilde 18: Kjørte “ftp” og ip-adressen. Skrev “anonymous” og “anonymous” fra ProFTPD sin dokumentasjon og fikk tilgang.

Skrev deretter ”ls” og fikk opp sensorveiledning for eksamen i 2023 og “welcome.msg”:

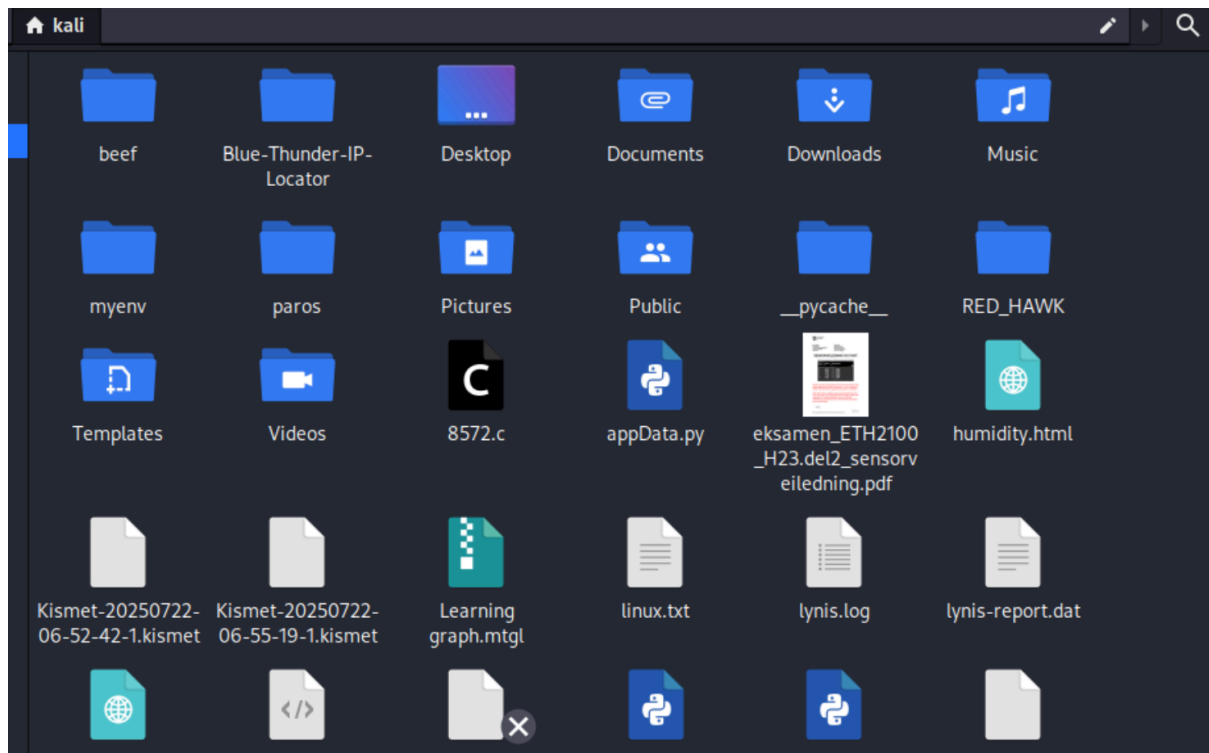
```
ftp> ls  
229 Entering Extended Passive Mode (|||34223|)  
150 Opening ASCII mode data connection for file list  
-rw-r--r-- 1 ftp ftp 486787 Oct 2 2023 eksamen_ETH2100_H23.del2_sensorveiledning.pdf  
-rw-r--r-- 1 ftp ftp 170 Aug 30 2021 welcome.msg
```

Figurbilde 19: Filene som ligger FTP-protokollen.

Deretter brukte en “get” på sensorveiledning filen, der den ble lagt til i “home” i kali linux, som vist i figurbilde 21:

```
ftp> get eksamen_ETH2100_H23.del2_sensorveiledning.pdf  
local: eksamen_ETH2100_H23.del2_sensorveiledning.pdf remote: eksamen_ETH2100_H23.del2_sensorveiledning.pdf  
229 Entering Extended Passive Mode (|||56194|)  
150 Opening BINARY mode data connection for eksamen_ETH2100_H23.del2_sensorveiledning.pdf (486787 bytes)  
475 KiB 19.87 MiB/s  
226 Transfer complete  
486787 bytes received in 00:00 (17.87 MiB/s)
```

Figurbilde 20: Overføring av senserveildningen til “home” mappen i kali



Figurbilde 21: Plassering av sensorveiledningen

Dette er innholdet i filen:



Neida, det ville vært ganske dumt hvis dere klarte å finne sensorveiledning og fasit til eksamen – inne i eksamens VMen. Det ville vært en ganske stor tabbe av foreleser...

Men denne filen er lagt her på en ikke-standard port for at de som gjør en grundig jobb skal finne den. Dette skal rapporteres i pentest rapporten som en KRITISK sårbarhet, og rapporteres som «Port 42420 eksponerer sensorveiledning».

Figurbilde 22: Lureri av foreleser, inkludert en forklaring på hvordan en skal dokumentere sårbarheten

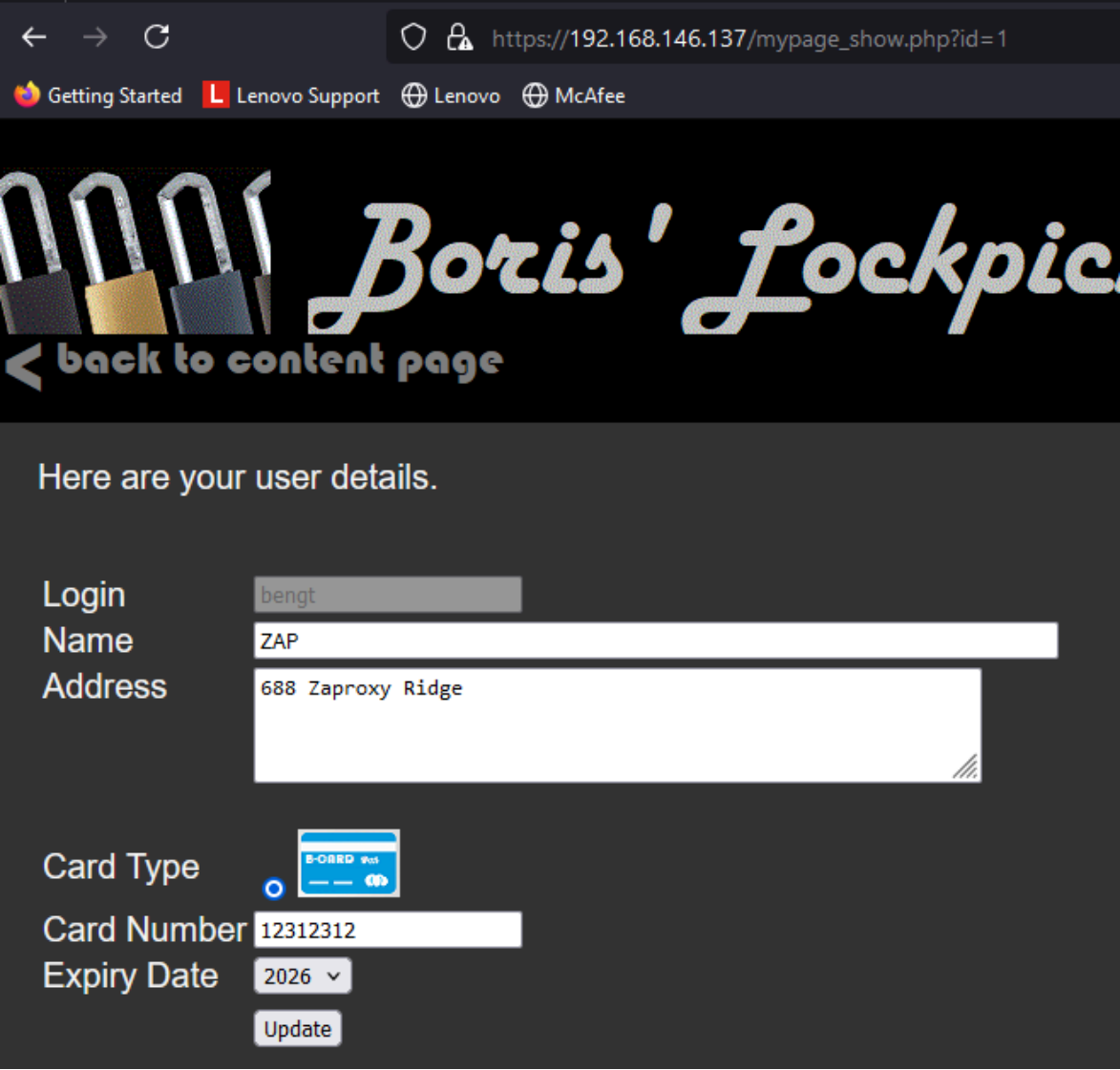
Dette er tydelig ikke den faktiske sensorveiledningen for eksamen som ble utført i 2023, som vist i skjermbildet over. Eieren av filen anerkjenner at de bruker en ikke standardport og at funnet bør bli rapportert. Det er også en skrivefeil i filen, ettersom det står at en skal rapportere dette som “Port 42420 eksponerer sensorveiledning”, men denne filen ble funnet i port 21.

Konsekvensen er minimale i denne situasjonen, men i realiteten, hvis en bedrift oppbeholder hemmeligheter som admin kredentailers, databaser for kunder (inkludert passordene deres) eller andre mer verdifulle filer, så kan en angriper lett få tilgang til disse og selge informasjonen eller be om løspenger fra bedriften, noe som svekker konfidensialitet og integritet til bedriften. Anbefaling blir dersom en ikke trenger å bruke en port i maskinen, så bør porten stenges, i hvert fall på en maskin som ikke sender filer til andre maskiner. Alternativt hvis maskinen sender filer, så bør man ikke bruke FTP, da filer som blir sendt over FTP er i plain tekst, uten noe form for kryptering.

En bør da bruke FTPS, som fungerer på samme måte som FTP, men inkluderer også SSL/TLS kryptering for filer som blir sendt gjennom protokollen. Alternativt kan man bruke SFTP for å sikre FTP koblingen, der SFTP bruker SSH for å kryptere og sikre filene i stedet for SSL/TLS. Uavhengig av oppsett for bruk av FTP protokollen, så bør admin forsikre at innlogging ikke bruker default brukernavn og passord, for å forhindre angriperne å få enkelt tilgang til FTP kanalen, da disse er lette å finne dokumentasjon på.

Sårbarhet 3	Webbasert Sårbarhet
Kritisk risiko	Insecure Direct Object Reference mot brukerid

Når en er logget inn i “mypage_show.php” siden av nettsiden, så viser nettsiden ID-en for den innloggede brukeren. Da var spørsmålet om en kunne endre på ID til en annen bruker som er registrert på nettsiden. Basert på informasjonen hentet fra SQL injection sårbarheten, spesifikt figurbilde 46, så har hver bruker en ID og deres assosierte informasjon som vist i figurbilde 23:

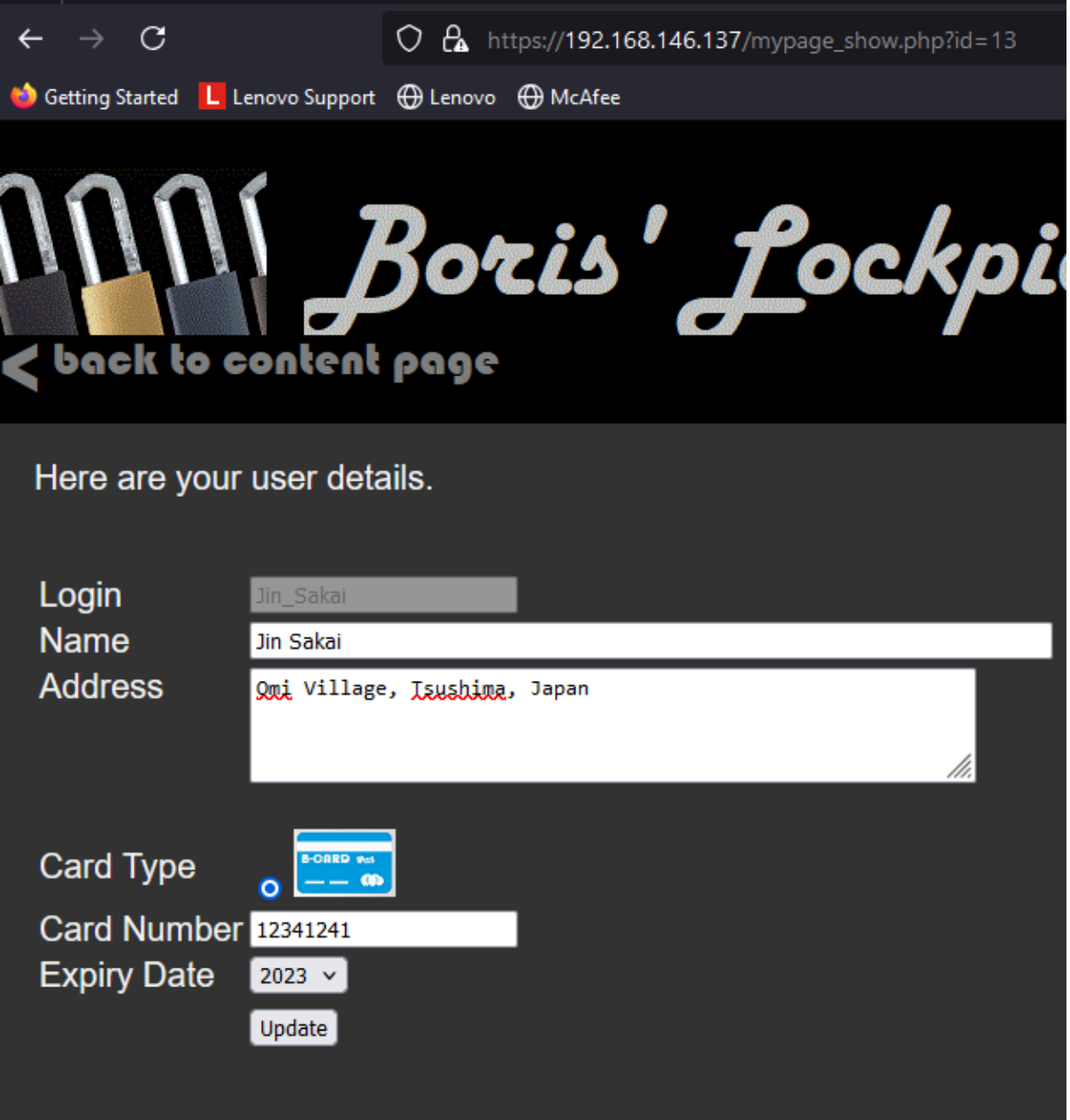


The screenshot shows a web browser window with the address bar displaying `https://192.168.146.137/mypage_show.php?id=1`. The browser's toolbar includes icons for 'Getting Started', 'Lenovo Support', 'Lenovo', and 'McAfee'. The page header features a graphic of three padlocks and the text 'Boris' Lockpick' in a stylized font, with a 'back to content page' link. The main content area is titled 'Here are your user details.' and displays the following information:

Login	bengt
Name	ZAP
Address	688 Zaproxy Ridge
Card Type	
Card Number	12312312
Expiry Date	2026
Update	

Figurbilde 23: Kundeinformasjonen som tilhører brukeren "bengt"

Ved å velge en hvilke som helst bruker fra de 370 brukerne, i denne situasjonen valgte en brukeren med id 13, får en opp kundeinformasjon til brukeren, uten at en trenger å autentisere seg selv på nytt:



← → ↻ https://192.168.146.137/mypage_show.php?id=13

Getting Started L Lenovo Support Lenovo McAfee

Boris' Lockpick

← back to content page

Here are your user details.

Login Name: Jin_Sakai

Name: Jin Sakai

Address: Omi Village, Tsushima, Japan

Card Type: 

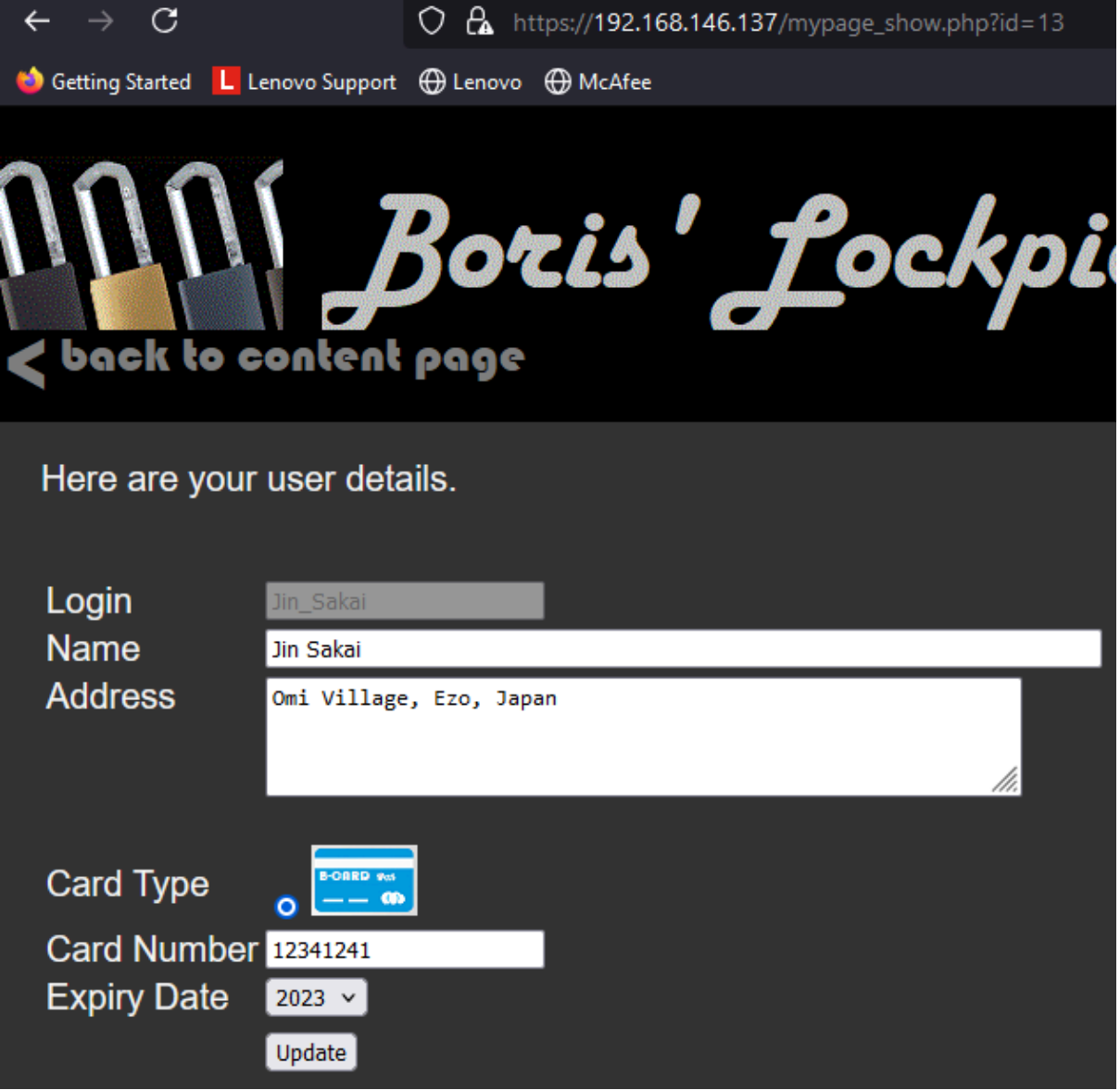
Card Number: 12341241

Expiry Date: 2023 ▼

Update

Figurbilde 24: Kundeinformasjon til bruker med ID=13

En kan modifisere innholdet og få det nye innholdet vist frem når man velger den samme ID på nytt. I denne situasjonen har jeg endret området fra “Tsushima” til “Ezo”. Når en velger ID=13 igjen, så vil den oppdaterte adressen blir værende som vist i figurbilde 25:

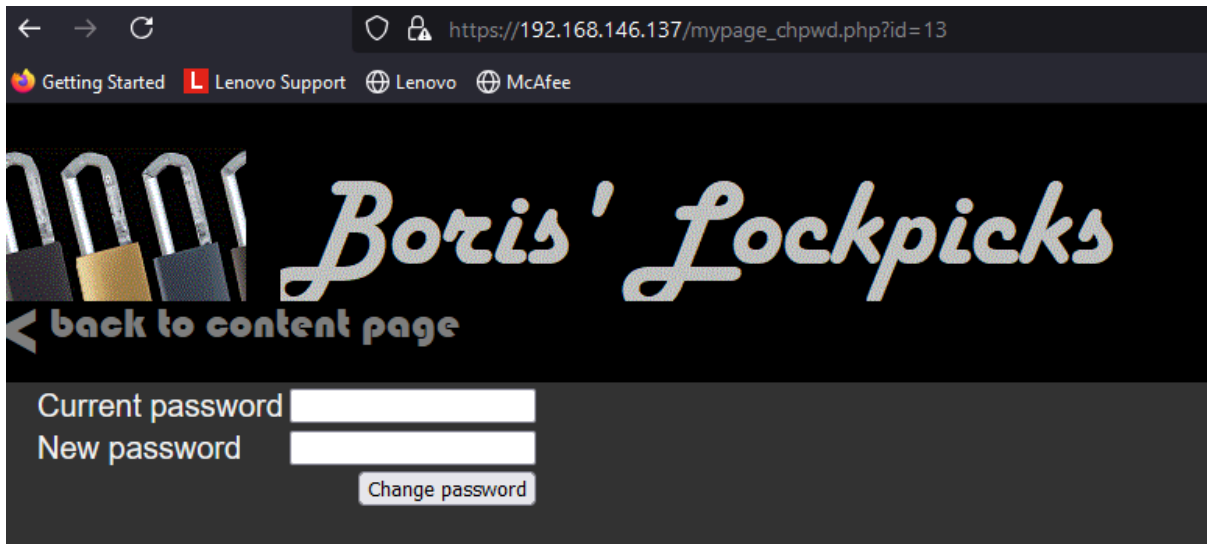


The screenshot shows a web browser window with the URL `https://192.168.146.137/mypage_show.php?id=13`. The browser's address bar includes navigation buttons and a security icon. Below the address bar, there are links for 'Getting Started', 'Lenovo Support', 'Lenovo', and 'McAfee'. The main content area has a dark background with a header featuring three padlocks and the text 'Boris' Lockpick'. A 'back to content page' link is visible. The main section is titled 'Here are your user details.' and displays the following information:

- Login Name:** Jin_Sakai
- Name:** Jin Sakai
- Address:** Omi Village, Ezo, Japan
- Card Type:** E-CARD (with a radio button selected)
- Card Number:** 12341241
- Expiry Date:** 2023 (with a dropdown arrow)
- Update:** A button to update the information.

Figurbilde 25: Oppdatert adresse for kunden "Jin Sakai"

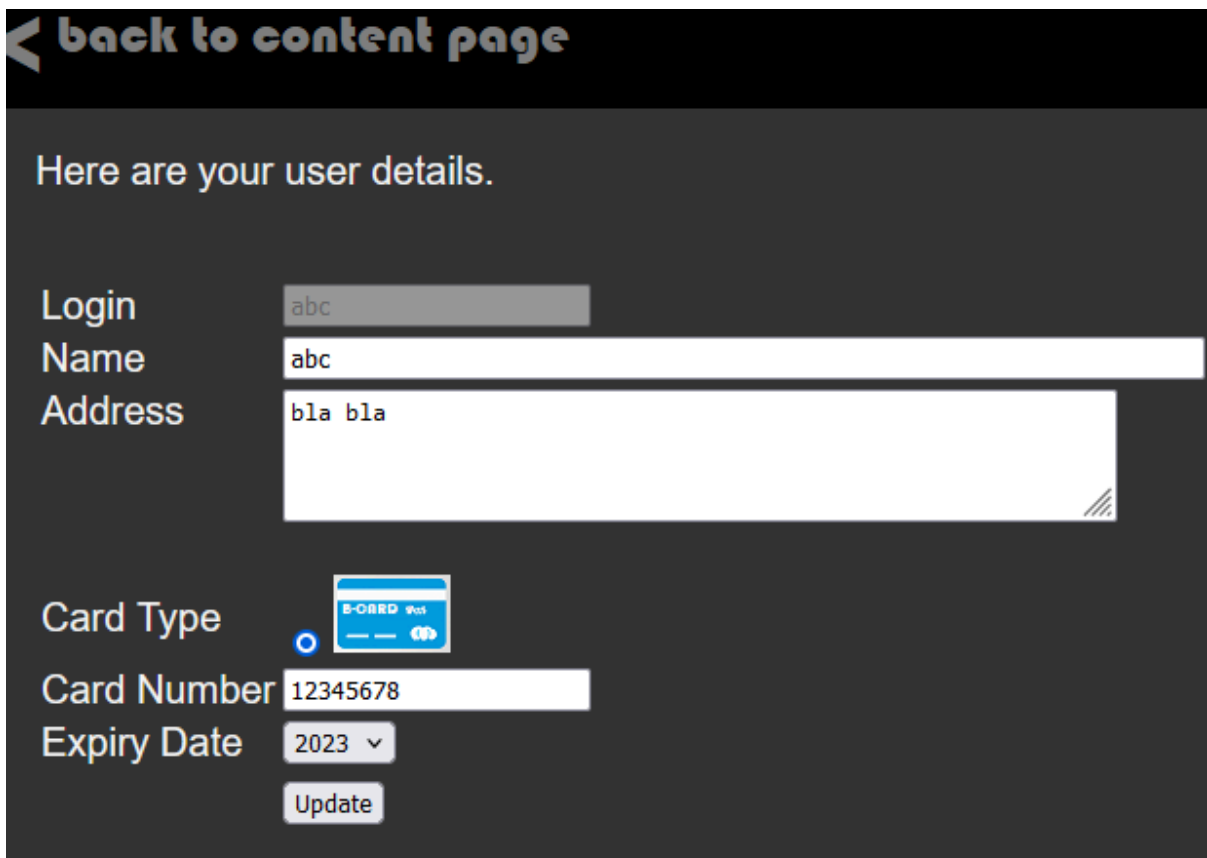
Det er også mulig for angriperen å oppdatere passordet, men da må en huske det gamle passordet først for så å legge til en ny passord for å utføre endringene. Denne falske accounten ble laget i 2023 under eksamen i etisk hacking, så en har dessverre glemt passordet for denne accounten:



The screenshot shows a web browser window with the URL `https://192.168.146.137/mypage_chpwd.php?id=13`. The page has a dark background with the title "Boris' Lockpicks" in a stylized font. Below the title is a "back to content page" link. The main form contains two input fields: "Current password" and "New password", followed by a "Change password" button.

Figurbilde 26: Endring av passord krever det gamle og nye passordet i korresponderende felter

For å demonstrere at det går an å impersonate som en bruker har en valgt brukeren "abc", hvor passord er også "abc":

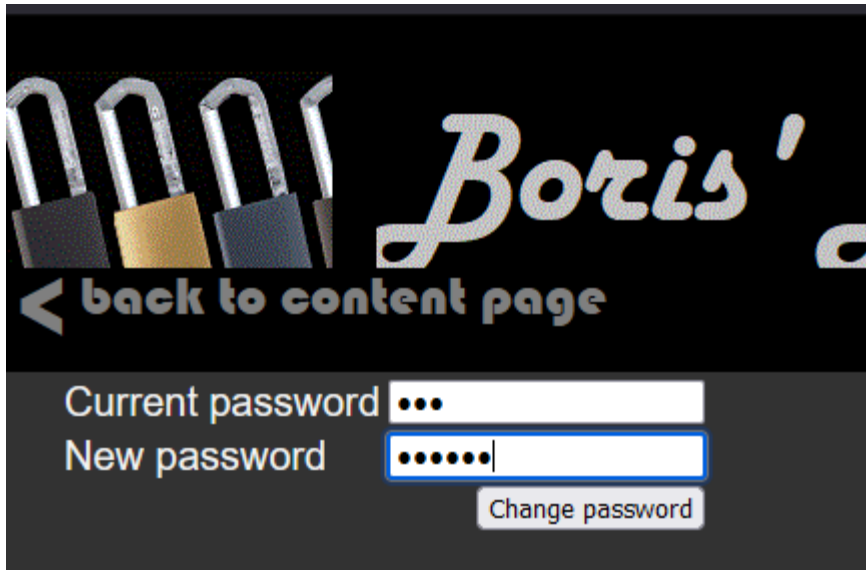


The screenshot shows a web page titled "back to content page" with the heading "Here are your user details." Below this, there are several form fields for user information:

- Login Name: abc
- Name: abc
- Address: bla bla
- Card Type: A radio button is selected next to a card icon labeled "B-CARD".
- Card Number: 12345678
- Expiry Date: 2023 (with a dropdown arrow)
- An "Update" button is located at the bottom of the form.

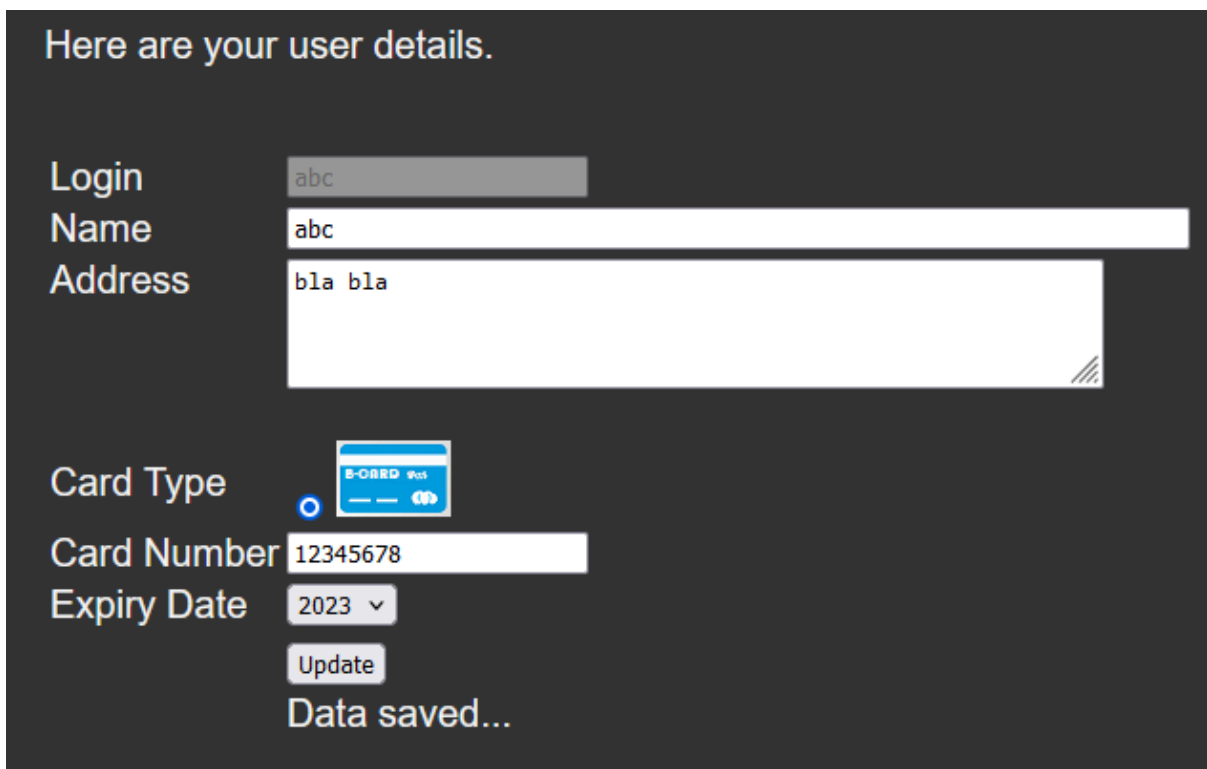
Figurbilde 27: Kundeinformasjon til brukeren "abc"

Skrev inn “abc” som gamle passord og ”abc123” som det nye passordet etter å ha valgt å endre på passordet:

The screenshot shows a dark-themed interface for changing a password. At the top, there is a header with three padlocks and the text "Boris'". Below the header is a button labeled "back to content page". The main form has two input fields: "Current password" with three dots indicating masked text, and "New password" with six dots indicating masked text. A "Change password" button is located below the "New password" field.

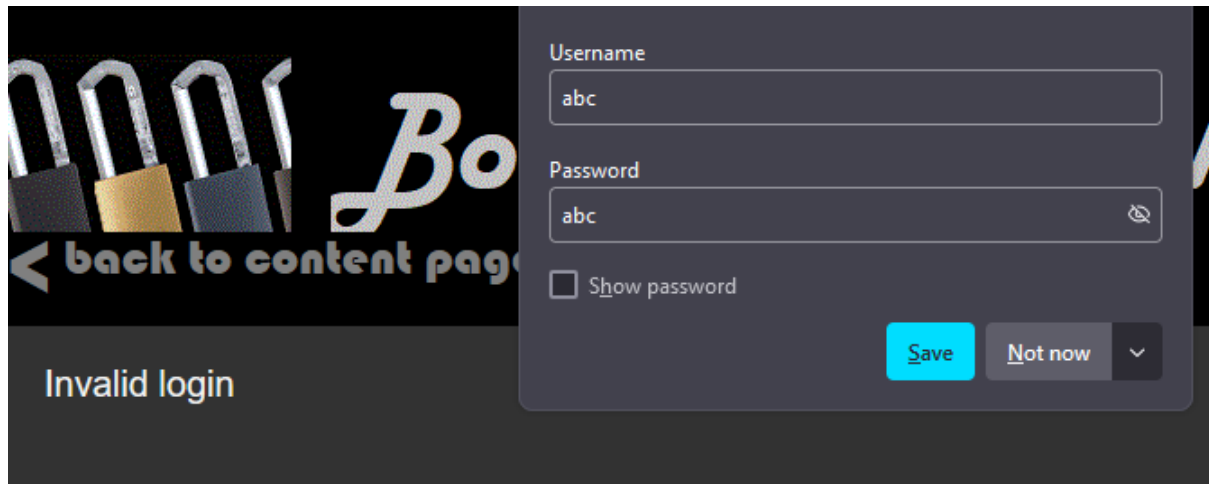
Figurbilde 28: Inntatt av det gamle og nye passordet til brukeren “abc”

Endringen ble da lagret:

The screenshot shows a dark-themed interface displaying user details. The text "Here are your user details." is at the top. Below it, the following fields are shown: "Login" with the value "abc", "Name" with the value "abc", and "Address" with the value "bla bla". There is a "Card Type" section with a radio button selected next to a "B-CARD" icon. Below this, the "Card Number" is "12345678" and the "Expiry Date" is "2023" with a dropdown arrow. An "Update" button is located below the expiry date. At the bottom, the text "Data saved..." is displayed.

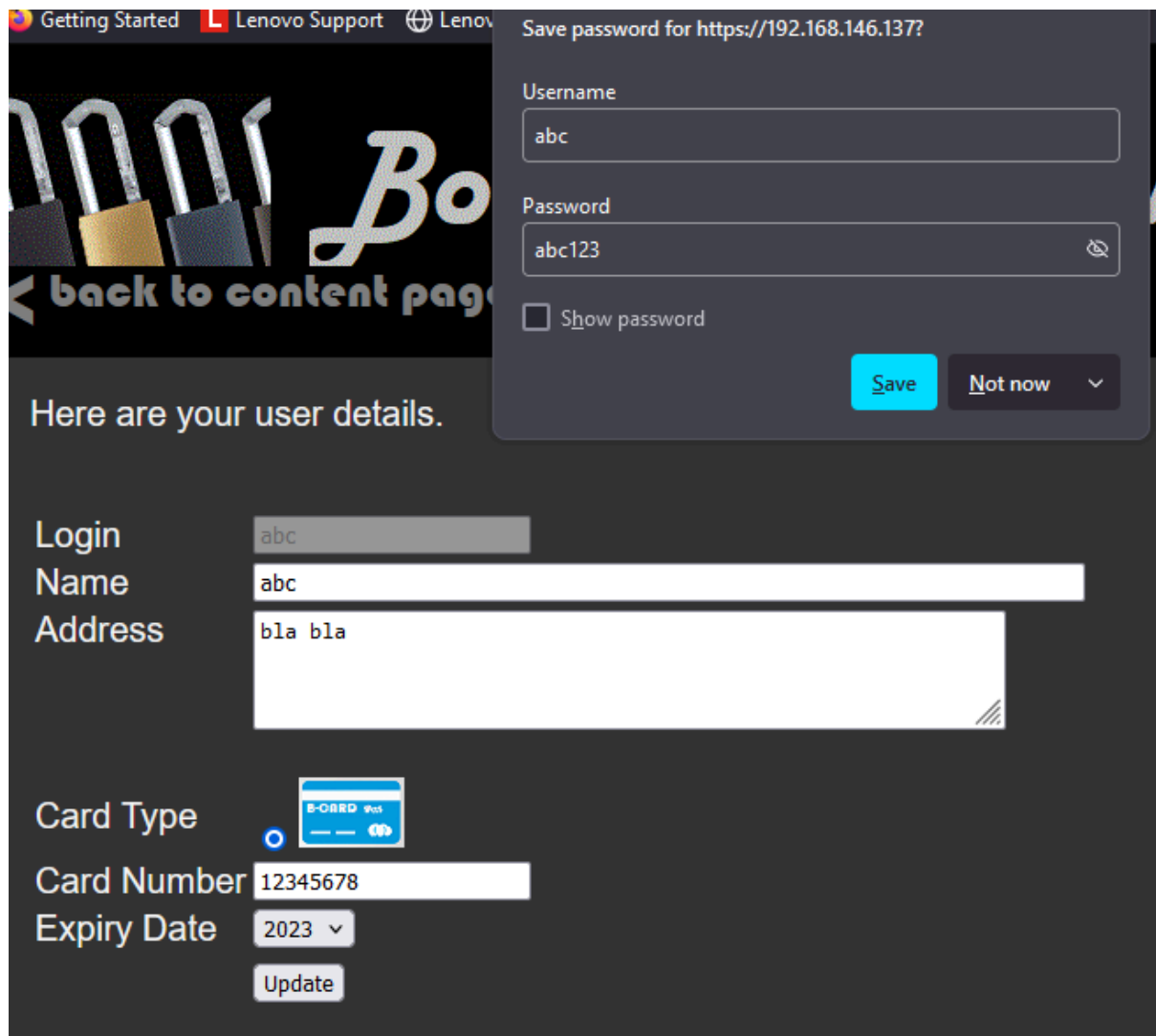
Figurbilde 29: Oppdaterte kundeinformasjonen, vist via “Data saved...” meldingen

Så når en prøver å logge som “abc” igjen med det gamle passordet, så får man en “Invalid login” melding:



Figurbilde 30: Resultat når en logger seg som abc med det gamle passordet

Login er bare gyldig når en legger inn det nye passordet “abc123”:



The screenshot shows a web application interface. At the top, there's a navigation bar with links like 'Getting Started', 'Lenovo Support', and 'Lenov'. Below this, there's a banner with the text 'back to content page'. A modal dialog is open, titled 'Save password for https://192.168.146.137?'. It contains fields for 'Username' (abc) and 'Password' (abc123), a 'Show password' checkbox, and 'Save' and 'Not now' buttons. Below the modal, the text 'Here are your user details.' is displayed. The user details form includes fields for 'Login Name' (abc), 'Address' (bla bla), 'Card Type' (with a dropdown menu showing 'E-CARD'), 'Card Number' (12345678), and 'Expiry Date' (2023). There is also an 'Update' button.

Figurbilde 31: Resultat når en logger seg som abc med det nye passordet

En har nå oppnådd full account access, samtidig som den legetime brukeren har mistet tilgang til kontoen, da passordet deres ikke lenger er det valide passordet i nettsiden.

Selvfølgelig, hvis dette var en admin konto, så ville angriperne ha flere muligheter, med tanke på tilgang til andre tjenester som bare admin har rettigheter til. Anbefaling er å verifisere at de autentiserte brukerne har autorisert tilgang og/eller om de kan modifisere data objekter.

Andre tiltak er å unngå å vise sensitive data i URLer, som ID-en i denne situasjonen.

Høy risiko:

Sårbarhet 4	Webbasert sårbarhet
Høy risiko	SQL-injection gjennom bruk av SQLmap

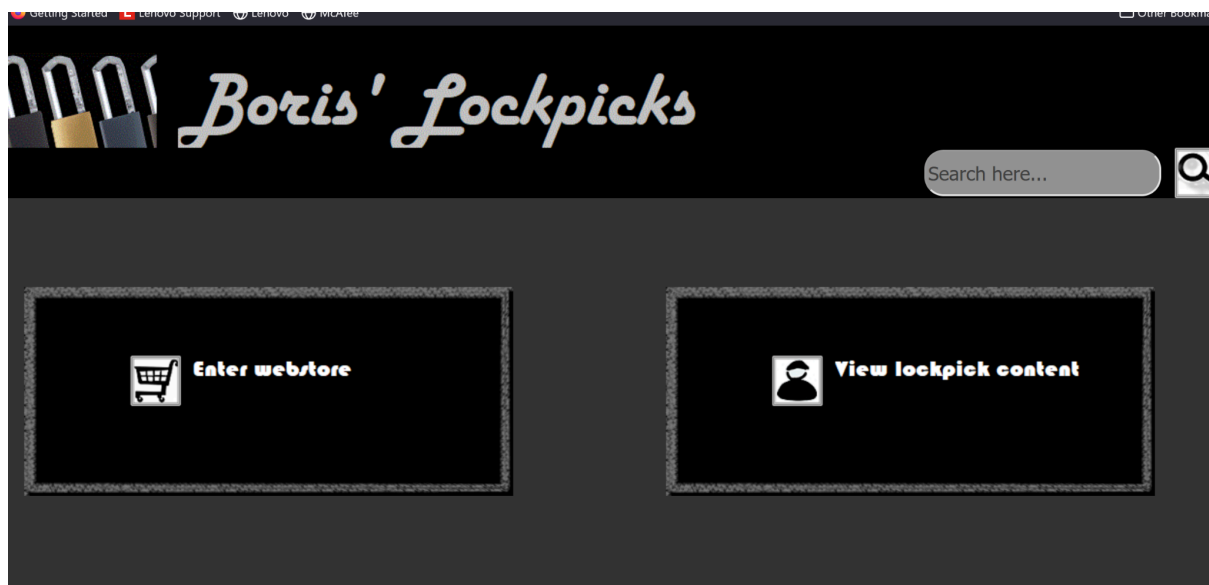
Varsel fra OWASP ZAP forteller om potensiell SQL-injection på nettsiden:

```
<html><head>
<title>Boris' Lockpick SUPERSALE</title>
</head>

<body bgcolor=#323232 leftmargin=0 topmargin=0 marginwidth=0 marginheight=0>
<p>En intern feil i SQL statementet<p>SQL = SELECT * FROM products WHERE uid = <p>Error - You have an error in your SQL
syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '' at line 1
```

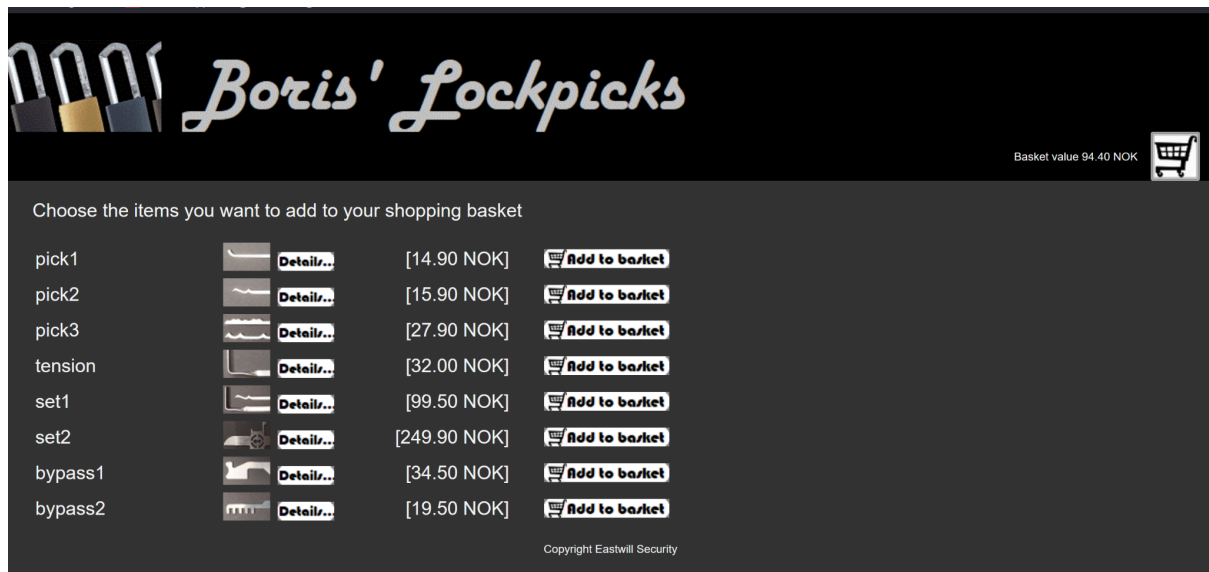
Figurbilde 32: Potensiell SQL-injection på nettsiden

Prøvde å lage en bruker i VM-en. I front-pagen trakk en på knappen “Enter webstore”:



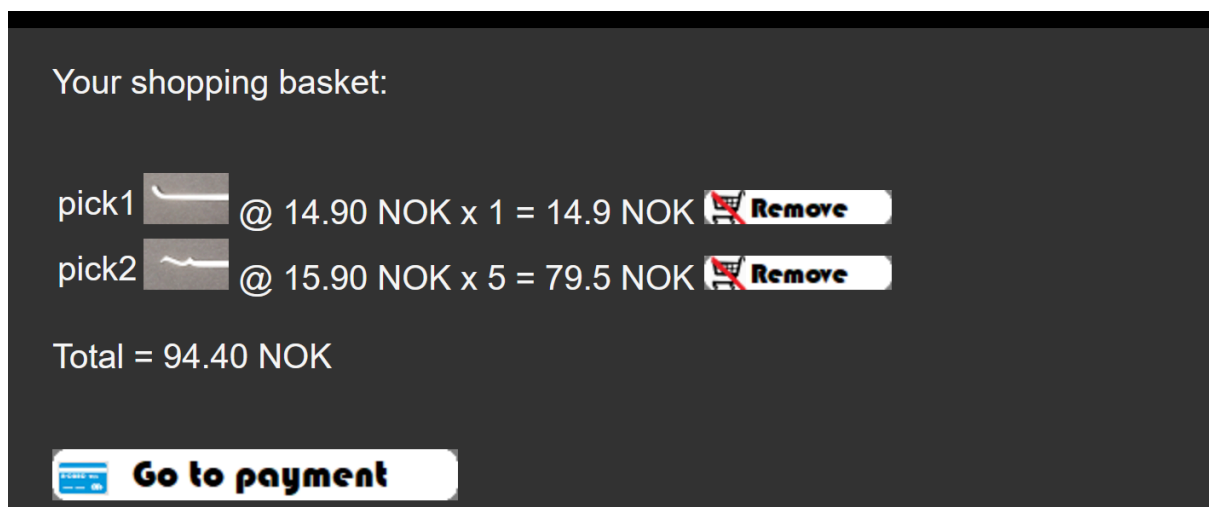
Figurbilde 33: Frontpagen til nettsiden

Valgte et par ting fra lista også trakk en på shopping-cart ikonet:



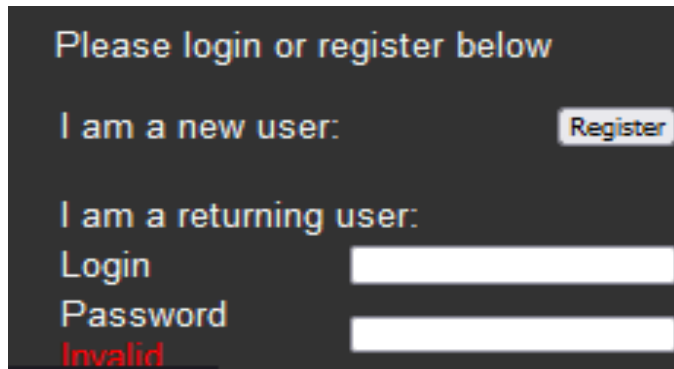
Figurbilde 34: Kjøp av ting og videre utsjekk når en trykker på shopping-cart ikonet

Trakk på "Go To Payment" knappen:



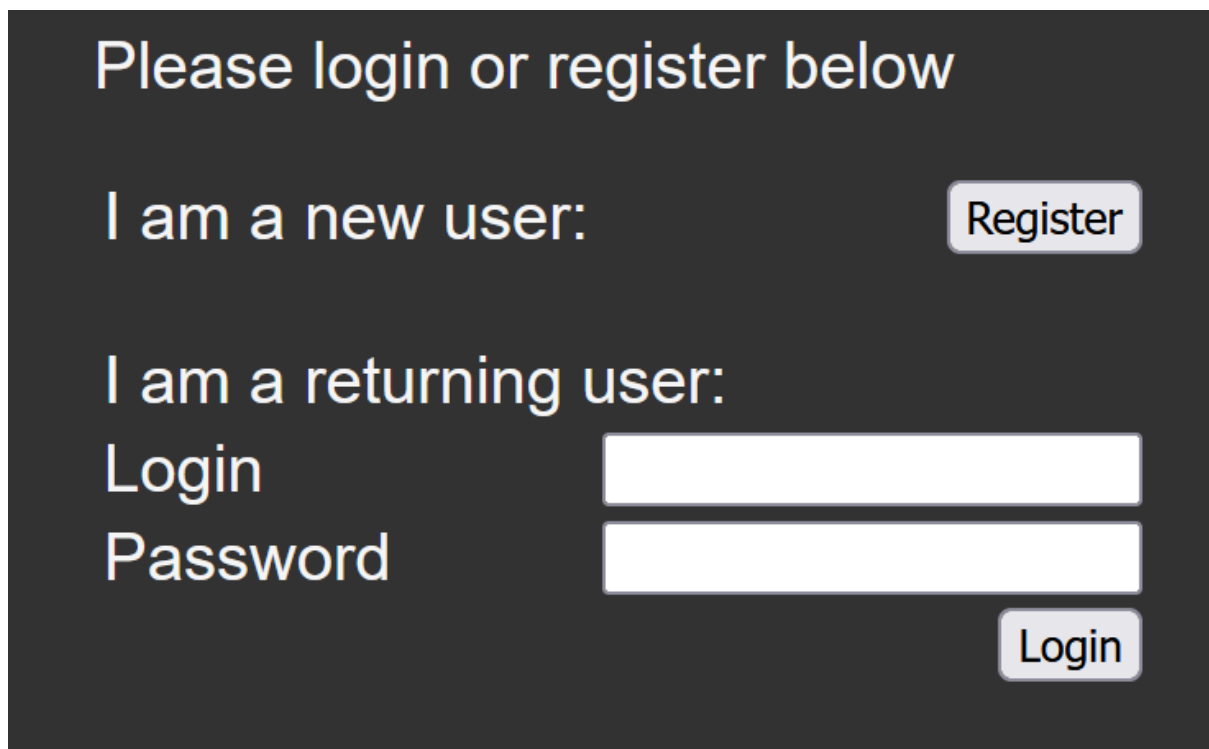
Figurbilde 35: Oppsummering av valgte varer og utsjekk

Ender opp i login-siden. En antar at en ikke har et gyldig passord og brukernavn, så når en prøver å skrive inn et hvilket som helst brukernavn og passord, så får en meldingen "Invalid" under "I am a returning user", som vist i figurbilde 36:



Figurbilde 36: Meldingen “Invalid” med rød farge kommer frem når en mislykket login er utført

Ved å trykke på “Register” knappen ved siden av “I am a new user”, så for en muligheten til å lage en konto på nettsiden, som vist i figurbilde 37:



Figurbilde 37: Bruker må registrere seg for å fullføre kjøpet eller logge inn med en eksisterende bruker i nettsiden

Fyller inn nødvendig informasjon, der username = "Ahmed_H" og password = "Jambo_123":

Fill in the following form with your details so we can complete the checkout process

NOTE TO STUDENTS: In reality this server in no way complies with GDPR or use your real password. To make sure no real creditcard information is provided has a 8 digit creditcardnumber... :-)

Name	Ahmed
Address	Møllefarete 44B
Card Type	☒ 
Card Number	12345621
Expiry Date	2026
Login	Ahmed_H
Password	••••••••
Confirm Password	••••••••
Create Account	

Figurbilde 38: Utfyllingskjema av kundeinformasjon før kjøp

Da har en laget en ny konto på nettsiden med kjøpsbekreftelse:

Your total value of this shopping basket is **\$94.40**

Payment will be made with the credit card currently on file with us. Its details are...

Card holder name	- Ahmed
Card number	- 12345621
Expiry date	- 2026

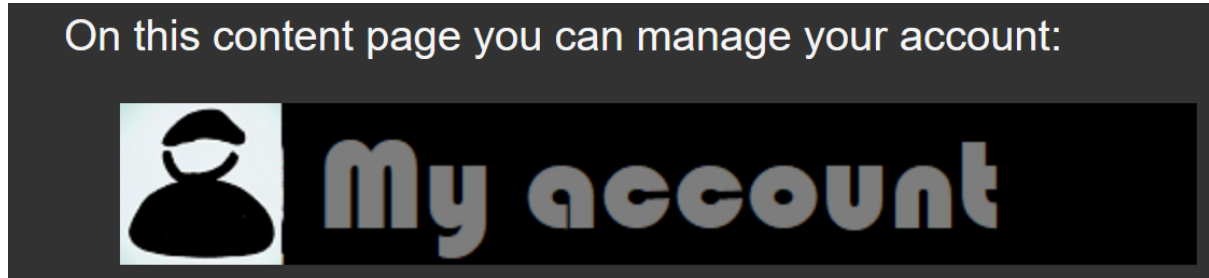
Delivery will be sent to the following address...

Møllefarete 44B

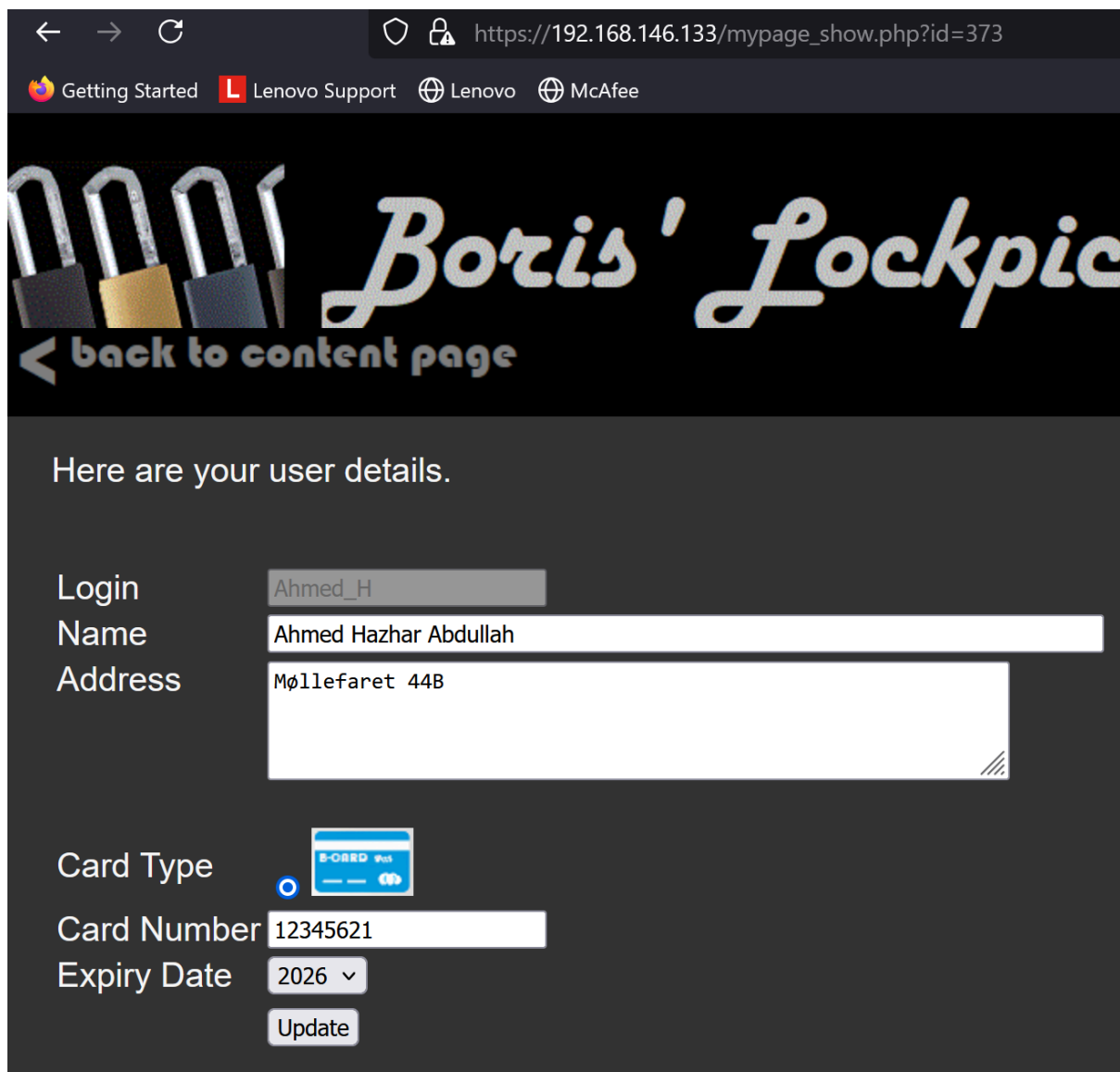
 **Complete payment**

Figurbilde 39: Kjøpsbekreftelse etter å ha laget en ny konto.

Etter litt research fant en ut at SQLMap fungerer nå som en har klart å lage en ny account. Alt en trenger er URLen som vises i “mypage_show.php” når en er logget inn som en valid bruker for å utføre angrepet og det finner man ved å trykke først på “My account” knappen mens en er logget inn, som vist i figurbilde 40 og 41:



Figurbilde 40: Informasjon av den nye brukeren ligger bak “My account” knappen



Figurbilde 41: Relevante URL-en som er nødvendig for å utføre sql injection via SQLmap

I kali linux kjørte en denne SQLmap kommandoen, hvor kommandoen lister alle tabeller som finnes i nettsiden, ved hjelp av URL-en:

```
(kali@kali)-[~]  
$ sqlmap -u 'https://192.168.146.133/mypage_show.php?id=373' --tables
```

Figurbilde 42: Bruk av SQLmap mot URL-en, der den skal liste alle tabeller

Dette er tabellene en får opp i databasen “borislockpicks”:

```
Database: borislockpicks  
[5 tables]  
+-----+  
| borislpbasket  
| customer  
| logon_sessions  
| lpbasket_entry_global  
| products  
+-----+
```

Figurbilde 43: Tabellene i “borislockpicks” databasen.

Deretter kjørte en denne kommandoen under mot den samme URLen. ‘-D’ er database target, i denne situasjonen “borislockpicks”, mens ‘-T’ er tabellen, der target er “customer”.

‘-dump’ dumper resultatet i en .csvs fil:

```
(kali@kali)-[~]  
$ sqlmap -u 'https://192.168.146.133/mypage_show.php?id=373' -D borislockpicks -T customer --dump
```

Figurbilde 44: SQLmap kommando for å dumpe alle brukere i tabellen “customer” i databasen “borislockpicks”

Dette er utdrag av resultatet, da det er rundt 380 brukere med passord i tabellen. Som vist i skjerm bilde 38, så er brukernavnet “Ahmed_H” og passordhashen til “Ahmed_H” tilgjengelig. Det står to “Ahmed_H” da en har tidligere jobbet med VM-en under eksamen. Siden det er rundt 380 brukere med passord, så har SQLmap limitert resultatet til de siste 256 radene:

352	LLWKVDV	WFKINAnS	8c0cad0737d10007a190e71023303a9	NiQoMuSf	CoguTECJ	2024
353	LLWKVDV	WFKINAnS	733ab868c48951750755337dcee24	NiQoMuSf	CoguTECJ	2024
354	LLWKVDV	WFKINAnS	9ada0dfcb4c5b5fec91f54b4b1b4a2e	NiQoMuSf	CoguTECJ	2024
355	LLWKVDV	WFKINAnS	0871811e2cfe3b25bbd7aef59295730b	NiQoMuSf	CoguTECJ	2024
356	LLWKVDV	WFKINAnS	903a98d709fa4683aaaa036b84c125a6	NiQoMuSf	CoguTECJ	2024
357	LLWKVDV	WFKINAnS	70c94c2d232730a95ddfd08ebfad178	NiQoMuSf	CoguTECJ	2024
358	LLWKVDV	WFKINAnS	0fa80a0e430aa62ee5f1dee164ba5b47	NiQoMuSf	CoguTECJ	2024
359	LLWKVDV	WFKINAnS	cb53cbe843579691474904134747d43f	NiQoMuSf	CoguTECJ	2024
360	LLWKVDV	WFKINAnS	d3bf7602cf7892ac674ac9471a1b47	NiQoMuSf	CoguTECJ	2024
361	LLWKVDV	WFKINAnS	eef278c9c2df465104a1af26ad26f765	NiQoMuSf	CoguTECJ	2024
362	LLWKVDV	WFKINAnS	6851b6ac45b3ef5b047ba5e8dbf699bb	NiQoMuSf	CoguTECJ	2024
363	LLWKVDV	WFKINAnS	a393035dab31ca7513d31dc229a34117	NiQoMuSf	CoguTECJ	2024
364	LLWKVDV	WFKINAnS	6c4ec228e79d1f61efb5751e70cf1b1	NiQoMuSf	CoguTECJ	2024
365	LLWKVDV	WFKINAnS	3ec47fbd5a2650rec94585659e1a3367	NiQoMuSf	CoguTECJ	2024
366	LLWKVDV	WFKINAnS	518ed29525738cebdac49c49e60ea9d3	NiQoMuSf	CoguTECJ	2024
367	LLWKVDV	WFKINAnS	26b17225b626fb9238849fd60eabdf60	NiQoMuSf	CoguTECJ	2024
368	LLWKVDV	WFKINAnS	93b885adfe0da089cdf634904fd59f71	NiQoMuSf	CoguTECJ	2024
369	LLWKVDV	WFKINAnS	b99834bc19bbad24580b3adfa04fb947	NiQoMuSf	CoguTECJ	2024
370	Witcher	Geralt of Rivia	3bbce48d0b8c69f434e3caa99e32dd40	Kaer Morhen, Northern Realms	12347282	2023
371	Sau	Ahmed	771ac9b8f0cfe4881ae4f65503863ccc	Sletteløkka 41	23413421	2023
372	Ahmed_H	Ahmed	c118d24b23d246c3351d4bc06bcc734b	Møllefaret 44B	12345621	2026
373	Ahmed_H	Ahmed Hazhar Abdullah	3a3151723278cc5e44644ccfa4dee237	Møllefaret 44B	12345621	2026

Figurbilde 45: Utdrag av alle 256 brukere, inkludert passordhashene til hver bruker i tabellen “customer”

Alternativt, siden en ønsker å få lista opp alle brukere og passord, så kunne en gå til denne filstien: “/home/kali/.local/share/sqlmap/output/192.168.146.133/dump/borislockpicks/” og åpne filen i mousepad:

```

1 uid,login,name,pwhash,address,cardnumber,expiryyear
2 1,bengt,Bengt Ostby,84d961568a65073a3bcf0eb216b2a576,Hoyskolen Kristiania\r\n0999 Oslo,12312312,2026
3 8,stian,Stian Kvals,9e43731b669b2e0f6accfc1881615efa,Gateadressen 12\r\n3299 Huttiheita,45645645,2024
4 9,abc,abc,900150983cd24fb0d6963f7d28e17f72,bla bla,12345678,2023
5 10,hehdbi,Jin Sakai,0192bbbbb9f7dc66e3b998329d16c289,omi coiwndfw+okfa\r\n0fnpod,28747293,2023
6 11,Sakai,Jin Sakai,f2e704bc0119e7edf9190360cce39246,"Omi village, Tushima, Japan",12355532,2023
7 12,Jin_Sakai,Jin Sakai,f2e704bc0119e7edf9190360cce39246,"Omi Village, Tsushima, Japan",12348291,2023
8 13,Jin_Sakai,Jin Sakai,f2e704bc0119e7edf9190360cce39246,"Omi Village, Tsushima, Japan",12341241,2023
...
365 370,Witcher,Geralt of Rivia,3bbce48d0b8c69f434e3caa99e32dd40,"Kaer Morhen, Northern Realms",12347282,2023
366 371,Sau,Ahmed,771ac9b8f0cfe4881ae4f65503863ccc,Sletteløkka 41,23413421,2023
367 372,Ahmed_H,Ahmed,c118d24b23d246c3351d4bc06bcc734b,Møllefaret 44B,12345621,2026
368 373,Ahmed_H,Ahmed Hazhar Abdullah,3a3151723278cc5e44644ccfa4dee237,Møllefaret 44B,12345621,2026

```

Figurbilde 46: Alle brukere og deres passorderhasher i mousepad

Disse er brukerne som ble funnet i databasen, inkludert passordhashene. En kan da ta passordhashen til “bengt” for eksempel og legge det i nettsiden “crackstation” for å knekke passordet:

Enter up to 20 non-salted hashes, one per line:

84d961568a65073a3bcf0eb216b2a576

I'm not a robot

This site is exceeding the reCAPTCHA Enterprise free quota.

reCAPTCHA

Privacy - Terms

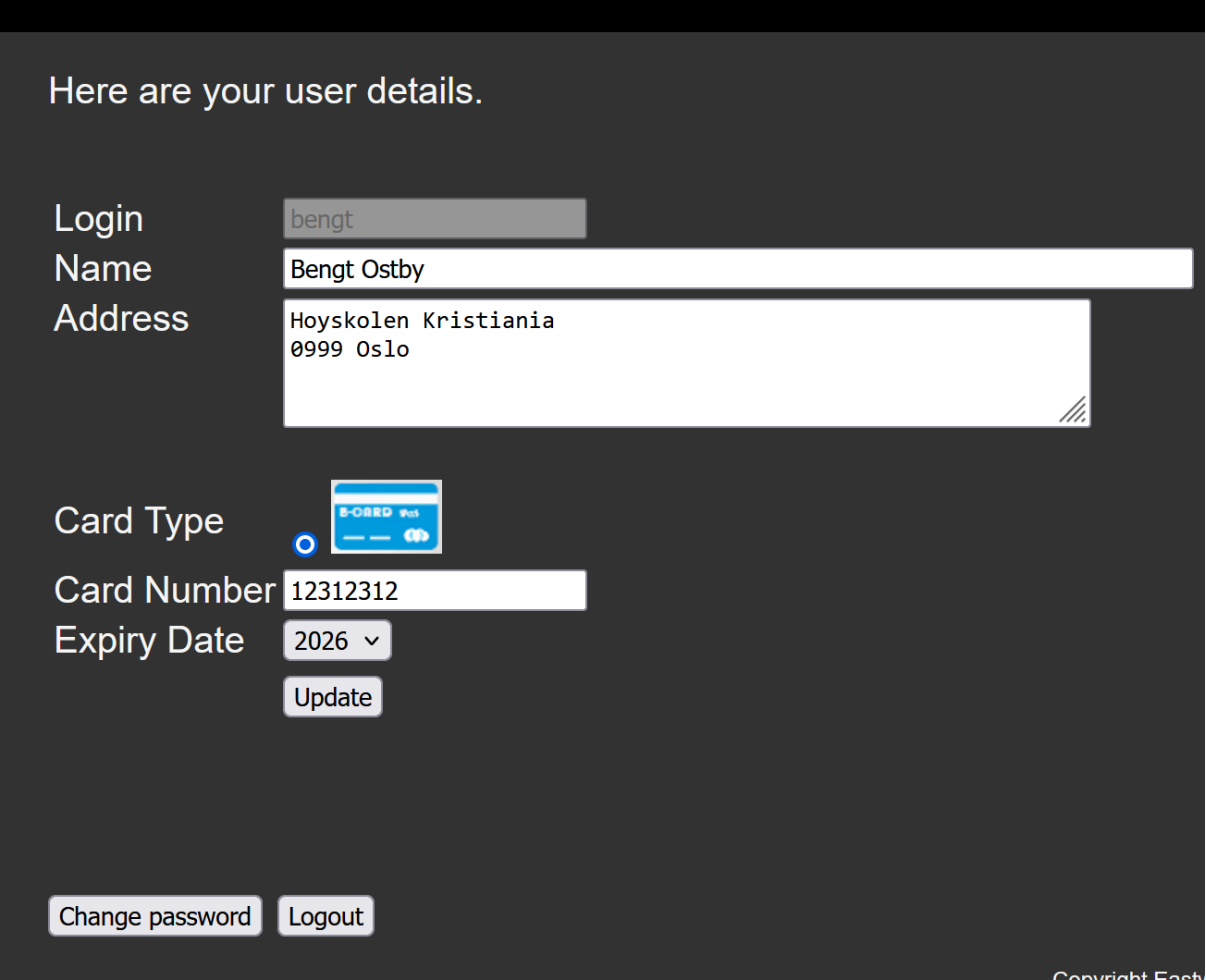
Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(sha1_bin)), QubesV3.1BackupDefaults

Hash	Type	Result
84d961568a65073a3bcf0eb216b2a576	md5	superman

Figurbilde 47: Plasserte passordhashen til brukeren “bengt”, dekrypterte hashen og fikk skrevet ut “superman” som passord i plaintext

Nå kunne en logge inn som “bengt” med passordet “superman” som crackstation har klart å knekke.



The screenshot shows a user profile interface on a dark background. At the top, it says "Here are your user details." Below this, there are several fields for user information:

- Login**: A text input field containing "bengt".
- Name**: A text input field containing "Bengt Ostby".
- Address**: A text input field containing "Hoyskolen Kristiania" and "0999 Oslo".
- Card Type**: A dropdown menu showing a blue card icon with "E-CARD" and "Pcs" text.
- Card Number**: A text input field containing "12312312".
- Expiry Date**: A dropdown menu showing "2026" with a downward arrow.

Below these fields is an "Update" button. At the bottom of the profile section, there are two buttons: "Change password" and "Logout".

Figurbilde 48: Innlogget som Bengt med passordet “superman” i nettsiden

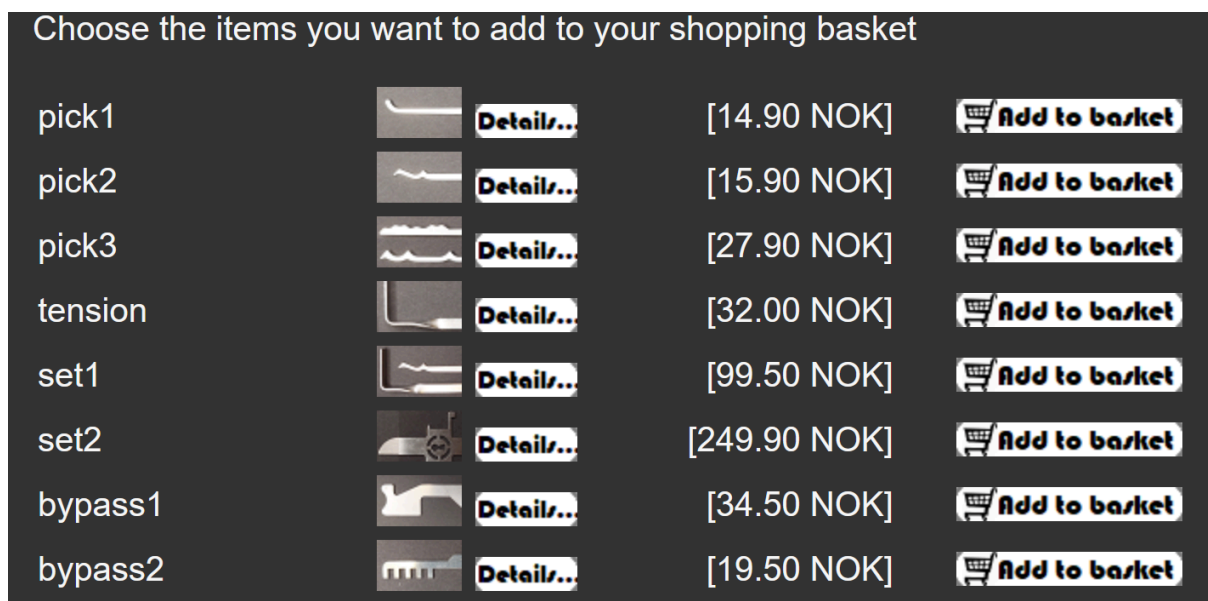
Konsekvensene her er at en SQL injection sårbarhet forårsaker tap av konfidensialitet og integritet. Angriperen kan essensielt spoofe identiteten til offeret og late som om de er en legetimt bruker i nettsiden, som vist i demonstrasjon over. De kan modifisere dataene, ødelegge data eller gjøre dataen utilgjengelig for brukerne. I verste fall så kan angriperne bli administratorer dersom de finner admin kredensialene.

Anbefaling er å forsikre at koden følger standard praksis for å unngå SQL-injection. Dette gjøres da gjennom bruk av “preparedstatements”, da det setter strenge krav for hvordan SQL spørringer skal oppføre seg. Viktig at kravene er satt riktig på plass da det å bare skrive inn “preparedstatement” uten korrekt konfigurering gjør at koden fortsatt er sårbar for angrepet. Da mener en at man skriver for eksempel i Java: “[preparedStatement.setInt](#)(1, nextID)” eller

“preparedStatement.String(2, nextString)”. Inkludert med det bør man også sanitere input-feltene ved å forhindre at brukeren kan legge til spesialtegn i innlogging feltene.

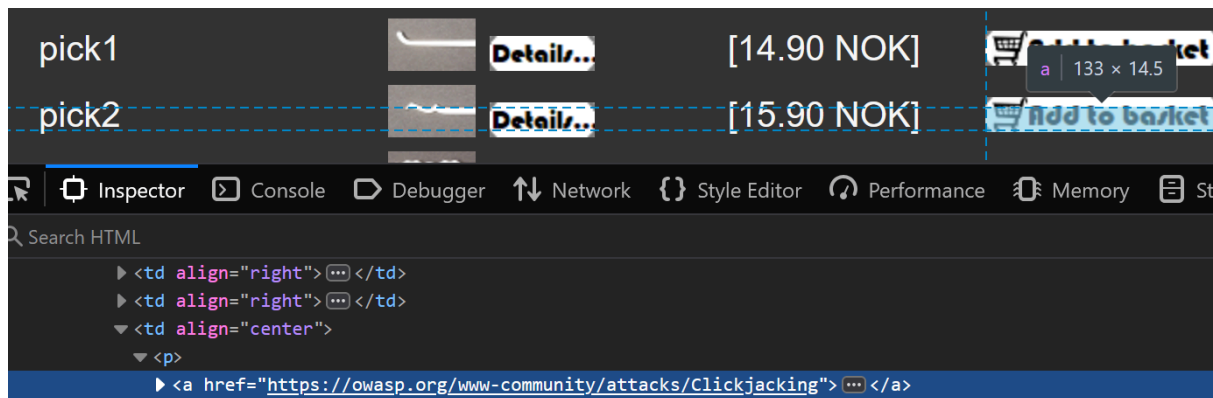
Sårbarhet 5	Webbasert sårbarhet
Høy risiko	Clickjacking mot knapper i siden

Gjennom testingen så fant en ut at knappene til kjøplisten i nettsiden kunne bli modifisert slik at når brukeren trykker på knappene, så blir de redirigert til den spesifikke URL-en som angriperne har plassert bak knappen. Et enkelt eksempel er i kjøplisten:



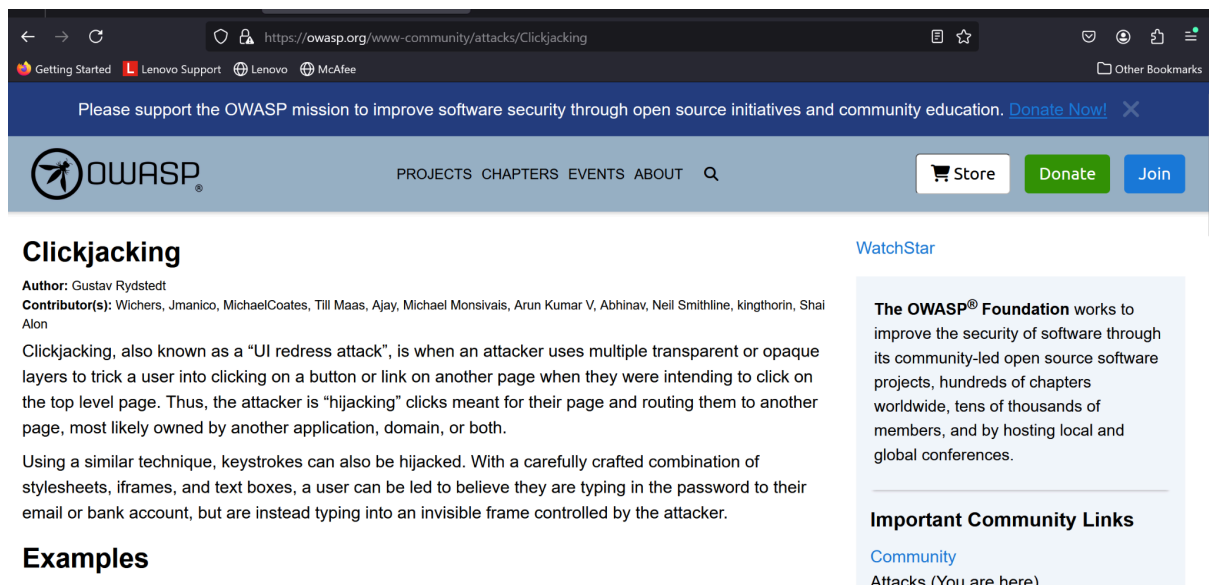
Figurbilde 49: Liste av alle items i butikken

Når en trykker på “Inspect” i siden, så kan en gå i "Inspector"-feltet og modifisere Javascript-koden. Her har en endret på URL-en til å være siden om “Clickjacking” fra OWASP.org, i stedet for funksjonene som legger til “pick2” i basket:



Figurbilde 50: Plassering av nettsiden om “Clickjacking” fra owasp.org i <a href> tagget for “pick2”

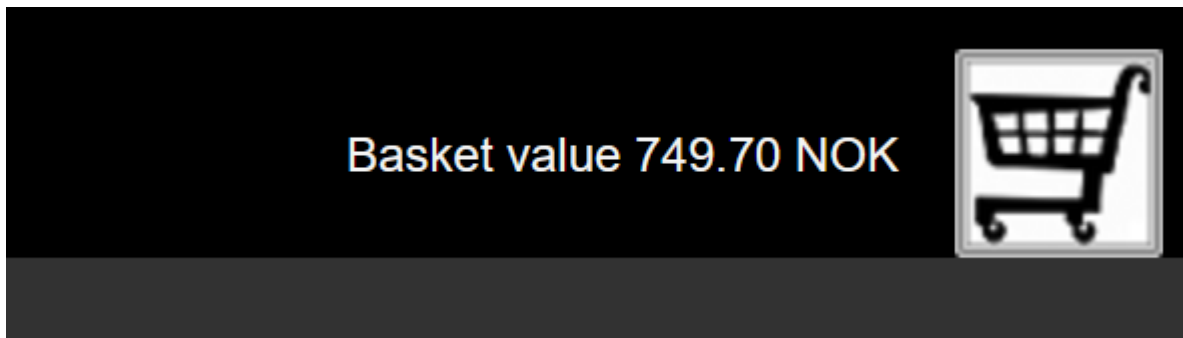
Når brukeren trykker på “Add to Basket” knappen for “pick2”, så blir de redirigert til owasp.org sin side om “Clickjacking”:



Figurbilde 51: [Owasp.org](https://owasp.org) sin side om Clickjacking etter å ha modifisert <a href> taggen

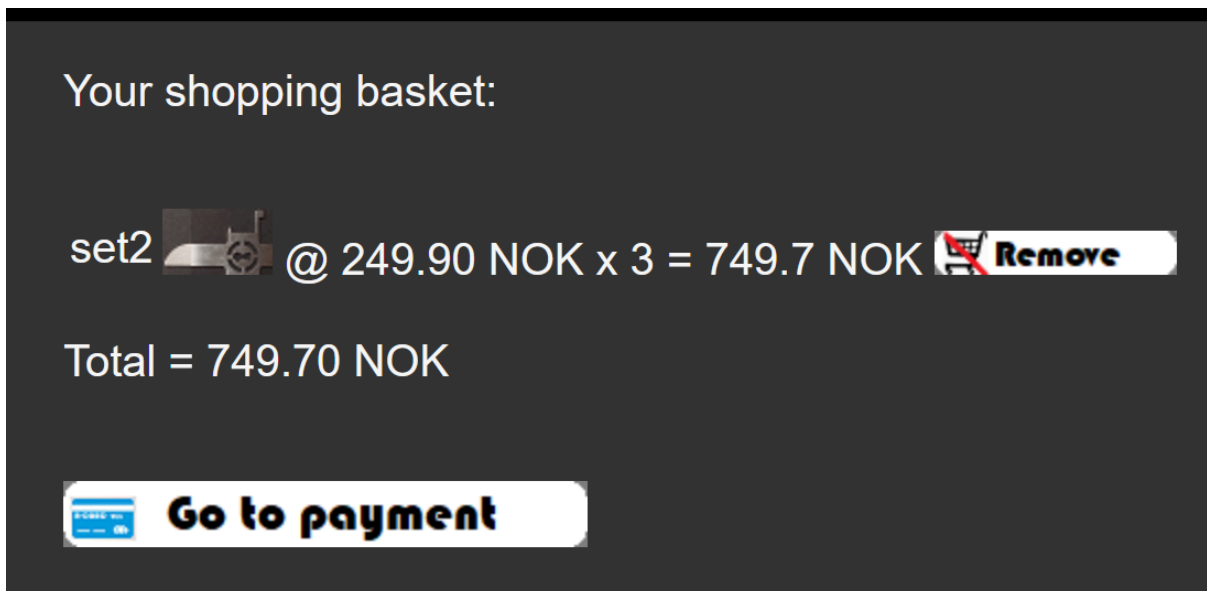
Dette er demonstrasjon av en enkel clickjacking angrep, der skadene er minimale, men det fremviser faren for slike angrep. Dersom en skulle faktisk demonstrere faren, så må en velge å kjøpe produktene først:

En trykker på “shoppingcart”-ikonet etter å ha valgt et par gjenstander:



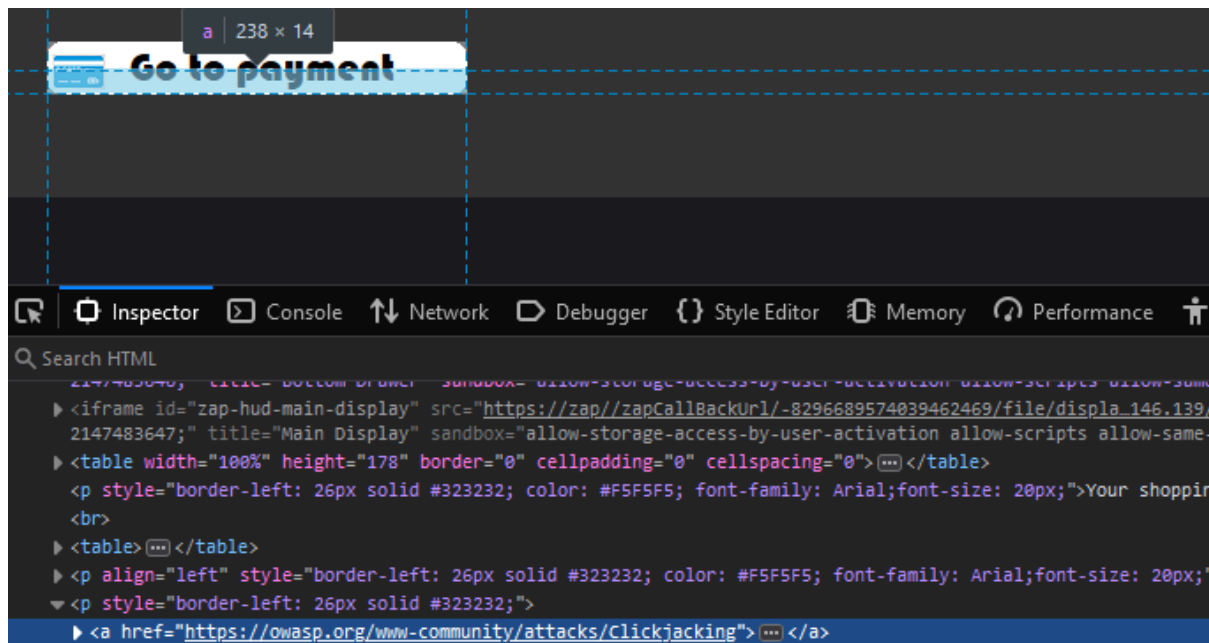
Figurbilde 52: Kjøp av varene som er lagret i shoppingcart

Trykker deretter på “Go to payment”:



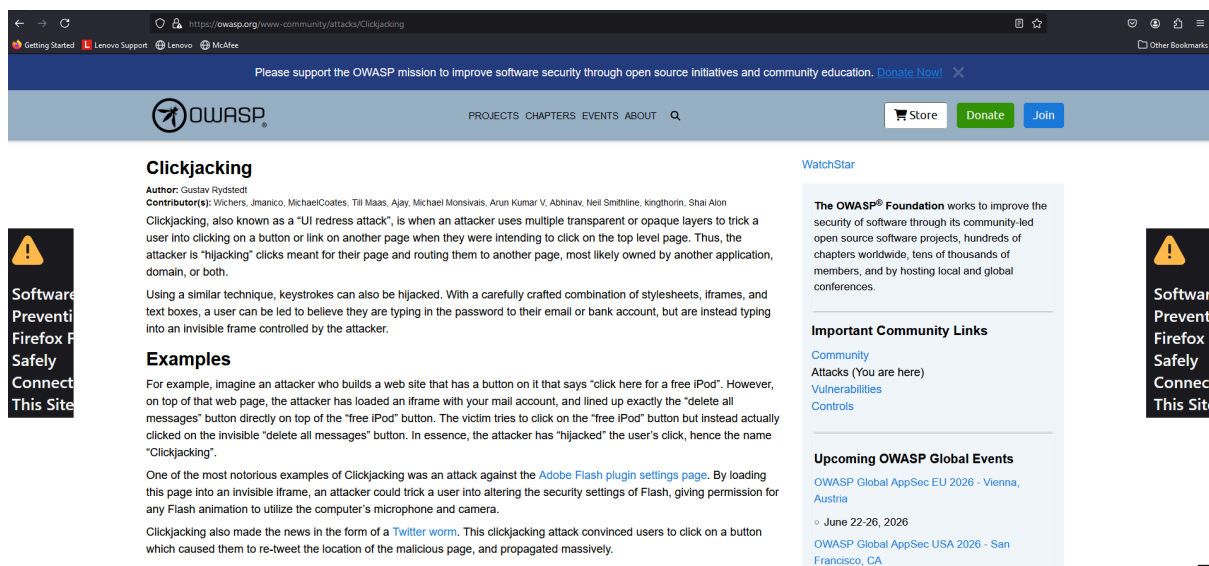
Figurbilde 53: “Go to payment” siden når en har valgt en gjenstand å kjøpe

En kan modifisere hyperlinken som er koblet til knappen. Bildet er ikke den peneste demonstrasjon da noe har gått galt med sertifikat verifikasjon i owasp zap og firefox:



Figurbilde 54: Erstatning av original url med url om clickjacking fra owasp.org, marker med blå bakgrunn

Når en trykker på knappen, så vil brukeren bli redigert til siden:



Figurbilde 55: Redirigering til owasp.org sin side om Clickjacking

Siden clickjacking angrepet baseres seg ut fra at brukeren blir redigert til siden som angriperen ønsker, ved trykke på UI-elementet som er modifisert, så er konsekvensene varierende. Angriperne kan laste ned malware på hostmaskinen, stjele kredensialer som er lagt til når brukerne logger inn eller når de lager en konto eller de kan få tilgang til lokasjonen

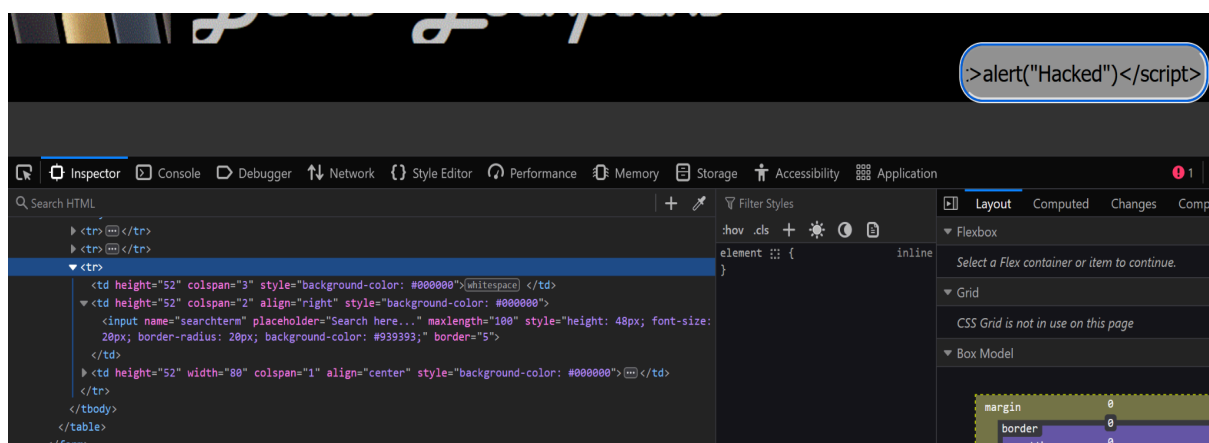
til offeret. Dette er bare noen få eksempler på en liste av flere konsekvenser som kan oppstå, hvor kreativiteten til angriperen er den eneste begrensingen.

Anbefaling er å implementere CSP frame-ancestors directive og sette det til “none”, da det forhindrer framing fra andre nettsider, som vist i demonstrasjon fra figurbilde 52-55.

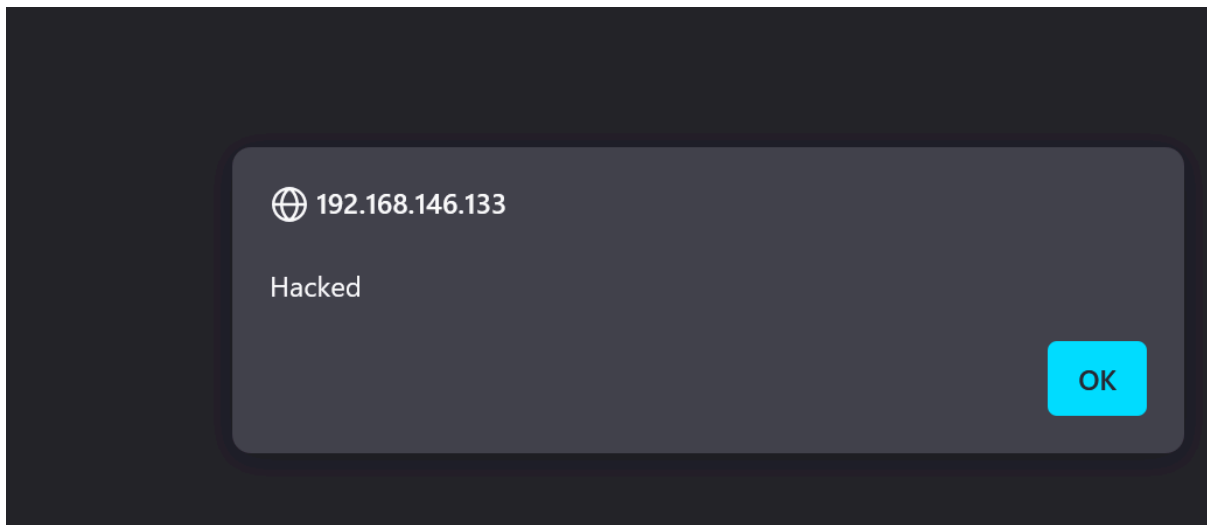
Implementasjon av SameSite cookie attributter og sette dem til “strict” eller “lax” er et ekstra tiltak, da cookien som tilhører en autorisert bruker ikke blir tilsendt til sider med iframe tagen i seg. En kan også kode javascript slik at siden ikke blir lastet ned i en <iframe> tag, kalt for “frame-busting”.

Sårbarhet 6	Webbasert sårbarhet
Høy risiko	Reflected XSS

Da nettsiden har en search funksjon i seg, lurte en om det var mulig å injecte javascript kode gjennom en Cross-site scripting angrep. I search funksjonen skrev en “<script>alert(“Hacked”)</script>” for å teste om sårbarheten er faktisk der. Et problem som oppstod var at en ikke kunne skrive hele XSS koden da feltet for search funksjonen hadde en karaktergrense på 30, men siden Javascript koden kan modifiseres, så økte en “maxLength” variabelen fra 30 til 100, som vist i skjermbilde under. Det tillot en å skrive hele javascriptkoden:



Resultatet er at alerten “Hacked” kommer frem når en trykker på enter:



Figurbilde 57: Alerten “Hacked” kommer frem når en trykker på enter

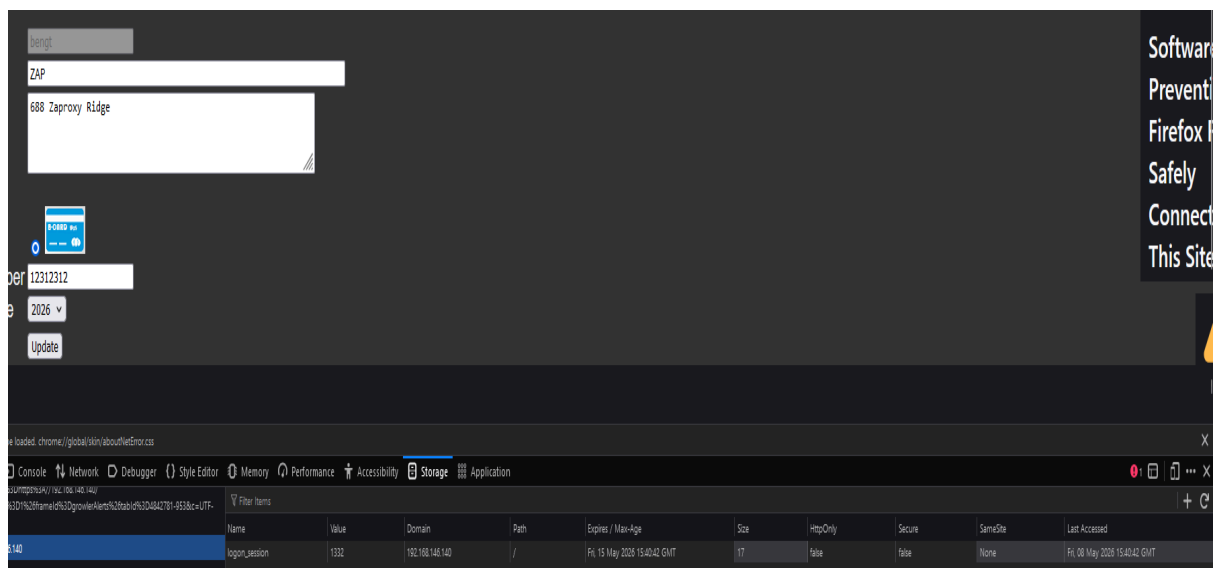
Slike angrep utnytter troverdigheten til nettsiden som nettleseren har, og siden nettleseren stoler på siden ettersom SSL/TLS sertifikatet matcher med root sertifikatet som er lagret til browseren, så slipper browseren å sjekke om det er lagt til noen scripts som en angriper har plassert i siden. Dette gjør at angriperne kan stjele session cookies og tokens og hijacke brukere, slik at de kan imitere dem. Dersom angrepet blir gjort mot en admin konto, så kan de angriperne lese kunders detaljer, få tilgang til databaser og modifisere koden på nettsiden. Angriperne kan eventuelt også stjele bank kredensialer, kontakt informasjon og kjøphistorikk, som vist i sårbarhet 4. Inkludert med det, så er det mulig for dem å legge til linker som laster ned malware på offerets maskin eller bruke kompromitterte kontoer som bots for videre bruk for DDoS angrep.

Anbefaling er å implementere “Output encoding” i Javascript koden. “Output encoding” er innebygget i HTML og Javascript, der oppsettet er forskjellige for begge. Uavhengig av oppsett, så innebærer bruken av “Output encoding” at en kan vise frem data som brukeren har skrevet inn på en trygg måte, da det forhindrer at variabler blir lest som kode i stedet for tekst i nettsiden. Sanitering av input-feltene er også et ekstra lag med beskyttelse, slik at man forhindrer at brukerne kan legge til `<script>` tags i feltene. Implementering av CSP, cookie attributter og firewalls er andre sikkerhets kontroller som bør bli lagt til sammen med output encoding for ekstra beskyttelse mot reflected XSS.

Sårbarhet 7	Webbasert sårbarhet
Høy risiko	Session hijacking

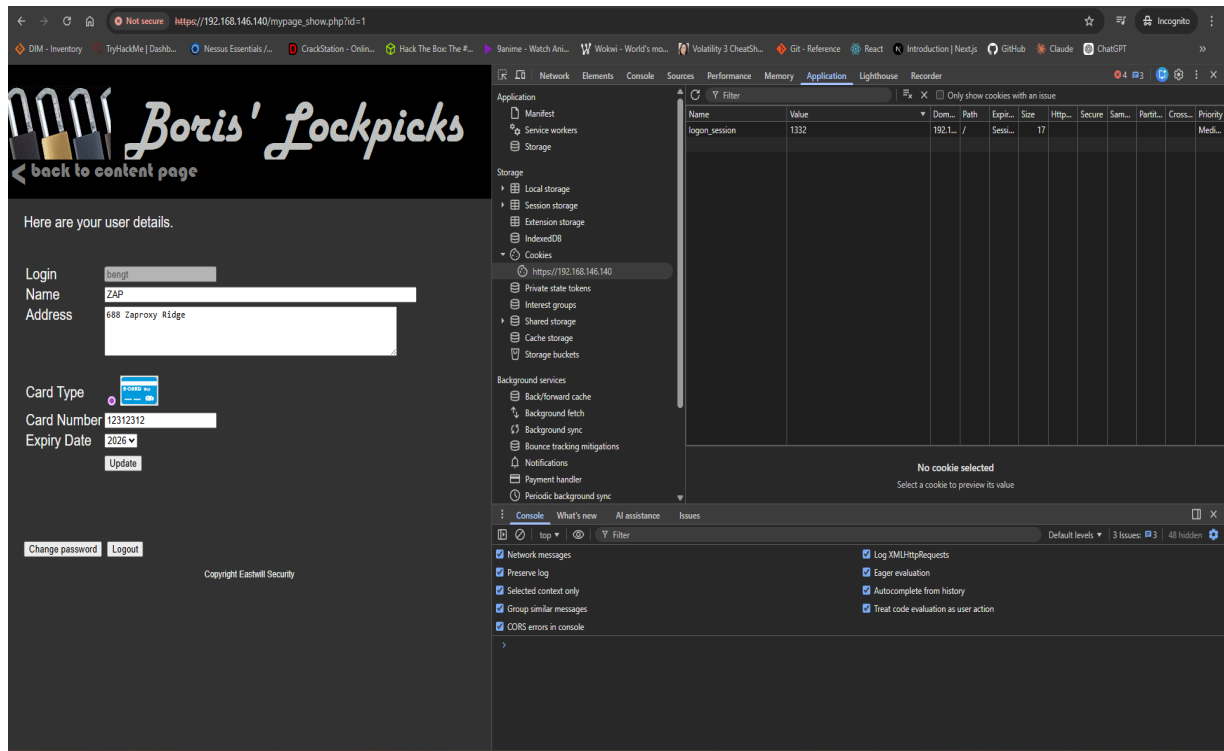
Basert på sårbarhet 3 så lurte en om det går an å utføre en liknende utnyttelse av nettsiden ved å misbruke session cookies. Normalt sett, så er hver bruker gitt en session cookie for midlertidig lagring av informasjon om brukerens preferanser og login status. Dette betydde at så lenge en behold en session aktivt så var det mulig å kopiere cookien i en annen nettleser:

En logget inn som “bengt” i firefox, der nettleseren tildelte en session cookie ved innlogging:



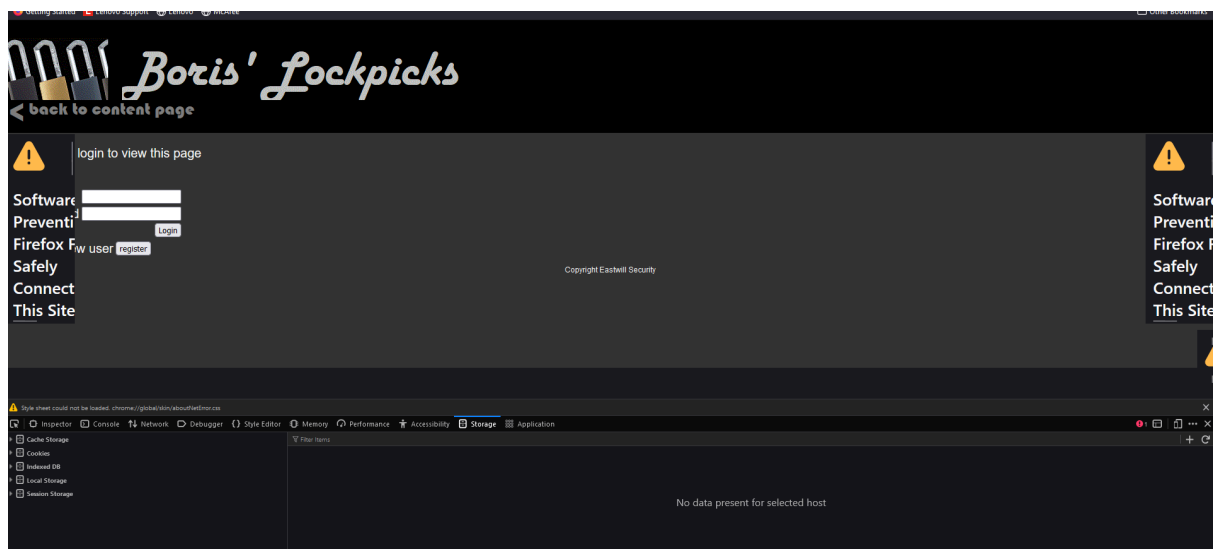
Figurbilde 58: Innlogget som valid bruker med tildelt session cookie

En kopierte da session cookie “Name” og “Value” over på Chrome sin “Cookies” vindue i inspectoren, hvor en allerede hadde nettsiden åpen og en var da utlogget. Fornyet siden etter handlingen, så ble en logget inn som “bengt”, uten at nettsiden ba om passord for autentifikasjon:



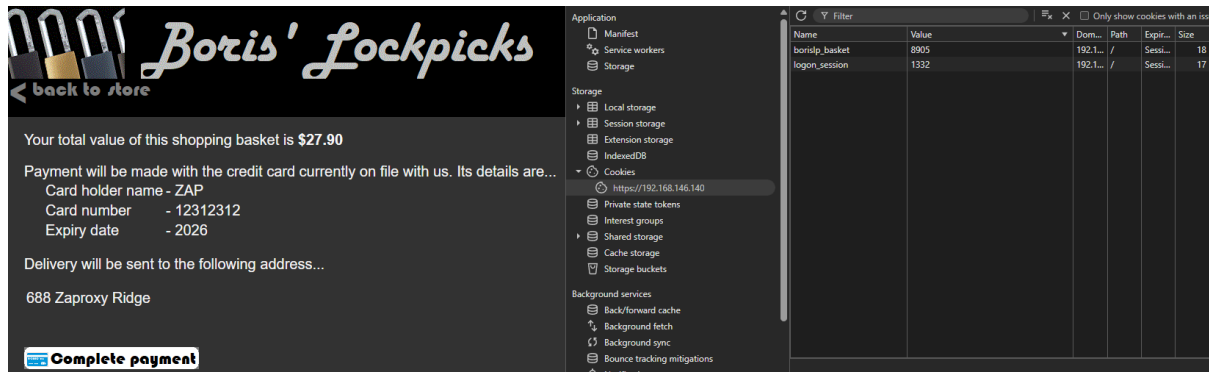
Figurbilde 59: Kopiering av cookie ledet til valid login, uten autentifikasjon fra chrome

Logget deretter ut i firefox for å teste om neste handling i chrome ville logge brukeren ut der og, da session cookien ble slettet vekk fra inspectoren:

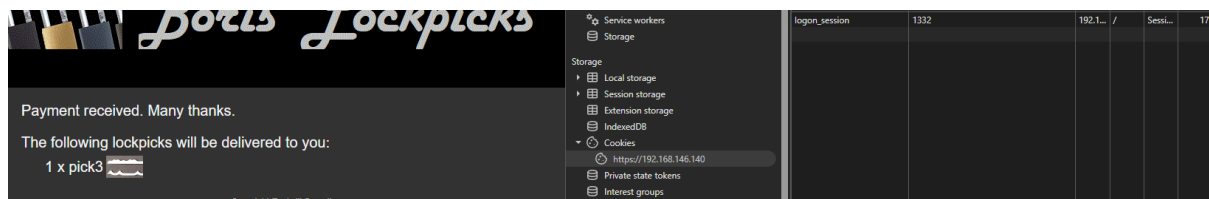


Figurbilde 60: Logget ut av siden i Firefox

En klarte å utføre et kjøp av et produkt i Chrome, selv om en originalt var utlogget i Firefox. I denne situasjonen klarte en å komme til oppsummeringslista av kjøp, før en fikk en bekreftelse på kjøpet av "pick3" som vist i figurbilde 62:



Figurbilde 61: Opsummeringslista etter å ha valgt “pick3” fra shopping siden



Figurbilde 62: Bekreftelse av kjøp for “pick3” i siden

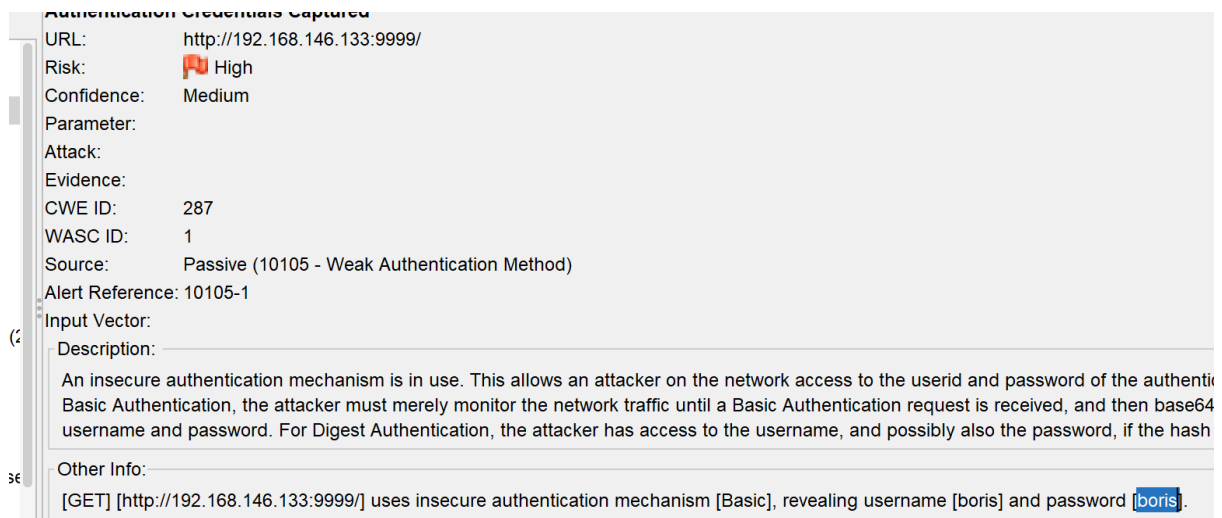
En har fortsatt mulighet til å utføre andre standard handlinger som en innlogget bruker som å endre på passord, lese gjennom PDF som ligger i “Lock Content” siden, legge til en kommentar eller sende en mail til en annen person. På lik linje med demonstrasjon av IDOR sårbarheten i sårbarhet 3, så kan en angriper late som de er den legetime brukerne ved å sniffe cookiene til brukerne over en usikkert nettverk og bruke cookiene for å utføre uautorisert handlinger, samtidig som offeret blir logget ut. Dette gjør at angriperne kan stjele identiteten til brukeren, forårsake økonomiske tap som vist over, der de også kan legge til malware i siden og utføre DDoS angrep mot web serveren.

Anbefaling er å implementere session reautentifisering hver gang en bruker logger seg ut, ved å enten be om brukernavn og passord på nytt og/eller at brukerne må fullføre en CAPTCHA, som er mer siktet mot bots, eller innlegging av en MFA som er mer siktet mot menneskelige angripere. Andre anbefalinger er å implementere monitorering av sessions, hvor en bruker kan få en beskjed fra browseren om en innlogging i en nettside, slik at de logger seg ut med engang for å forhindre mer skader og selvfølgelig endre på kredensialene for fremtidig bruk.

Medium risiko:

Sårbarhet 8	Webbasert sårbarhet
Medizum risiko	Nettsiden har svake sikkerhetsmekanismer mot brute-force angrep i port 9999

OWASP ZAP flagget port 9999 som en høy risiko sårbarhet. Port 9999 som vist i figurbilde 5 og 8 er porten som Abyss web server kjører på:



Figurbilde 63: OWASP ZAP markering av port 9999 som høy sårbarhet, der den viser potensiell brukernavn og passord til nettsiden.

Prøvde å skrive “boris” og “boris” inn i prompten, men det fungerte ikke. Da tenkte en at kanskje Hydra klarte å løse problemet. Testet mye forskjellige: Prøvde å teste mot URLen i helhet og det fungerte ikke, da Hydra ikke visste hva slags type service URL-en kjører (se figurbilde 64). Testet mot http-get og det fungerte, men så fikk en meldingen “file for logins not found: boris” (se figurbilde 65). Til slutt kom en til konklusjonen at “rockyou.txt” filen er det beste valget en har så den ble lagt til som wordlist istedet, der problemet lå i parameterne, spesifikt at en brukte “-l” i stedet for “-L”. Ved å bruke “-L”, fikk en til slutt en matchende passord, “tinkerbell” til brukernavnet “boris” som vist i figurbilde 66:

```
(kali@kali)-[~]
$ hydra -L admin -P /usr/share/wordlists/rockyou.txt http-get http://192.168.146.133:9999
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 10:06:17
[ERROR] Unknown service: http://192.168.146.133:9999

(kali@kali)-[~]
$ hydra -L admin -P /usr/share/wordlists/rockyou.txt http-get http://192.168.146.133:9999/
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 10:06:26
[ERROR] Unknown service: http://192.168.146.133:9999/
```

Figurbilde 64: Mislyktes hydra brute-force mot url med brukernavn som admin

```
(kali@kali)-[~]
$ hydra -L boris -P /usr/share/wordlists/rockyou.txt http-get://192.168.146.133:9999/
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 10:10:03
[ERROR] File for logins not found: boris
```

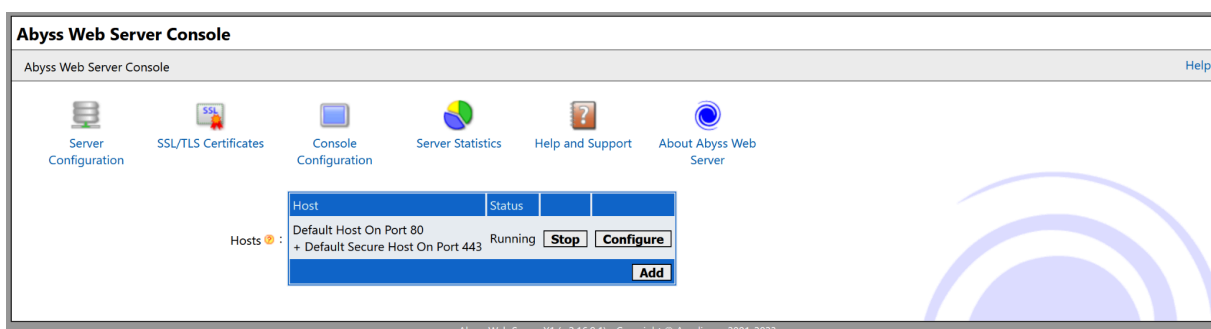
Figurbilde 65: Mislyktes hydra brute-force da Boris ikke var funnet i filen.

```
(kali@kali)-[~]
$ hydra -L boris -P /usr/share/wordlists/rockyou.txt http-get://192.168.146.133:9999/
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-03-30 10:13:26
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking http-get://192.168.146.133:9999/
[9999][http-get] host: 192.168.146.133 login: boris password: tinkerbell
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-03-30 10:13:27
```

Figurbilde 66: Valgte filstien til “rockyou.txt”. Passordet til “boris” var “tinkerbell”

Når en skriver inn “boris” som brukernavn og “tinkerbell” som passordet, så blir man logget inn i nettsiden “Abyss Web Server Console”:



Figurbildet 67: Siden til “Abyss Web Server Console”

Figurbilde 68 viser server root directoriet som hele nettsiden kjører på:

Parameters

Abyss Web Server Console :: Server Configuration :: Parameters

Server Root :

Timeout : second(s)

Maximum Simultaneous Connections :

Advanced Parameters :

Operating System Parameters :

Back-end Operation Support :

Figurbilde 68: Server root directory “/home/boris/apps/abyssws”

Metodologen demonstrerer hvordan en angriper kan få tilgang til Web server konsollen, når nettsiden mangler autentiseringssystemer og når servicen bruker passord som er lett å brute force. Dette gjør at angriperen kan utføre brute force angrep gjennom verktøy som hydra, og få tilgang til siden. Anbefaling er å legge til et ekstra sikkerhetslag gjennom en 2-faktor autentifikasjon, da angriperen er nødt til å gå forbi dette også, noe som oftest krever mer tid og ressurser å gå forbi.

Inkludert med det, så bør nettsiden også legge til en delay på fem min eller mer etter x antall login, slik at det gjør det upraktisk for angripere å bruke verktøy som hydra, da den tester mot flere hundre passord i kort tid. Til slutt bør ikke siden bruke passord som er lett å brute force og heller bruke passord med mer kompleksitet, hvor en kombinerer store og små bokstaver, tall og spesielle tegn i passordet. Denne sårbarheten tillater angriperne også å få oversikt over filer på root directory, men kan ikke modifiserte data. Dersom angriperen finner en måte å modifisere data på, så øker risikovurdering til kritisk.

Side 45

Sårbarhet 9	Webbasert sårbarhet
Medium risiko	Utsatt skjult side på port 42420

Fra figurbilde 7 og 8, så var det også oppdaget en åpen port på 42420. Dette er en port som oftest ikke blir brukt i web applikasjoner, så da en skrev inn: “192.168.146.133:42420” så kom denne siden fram:

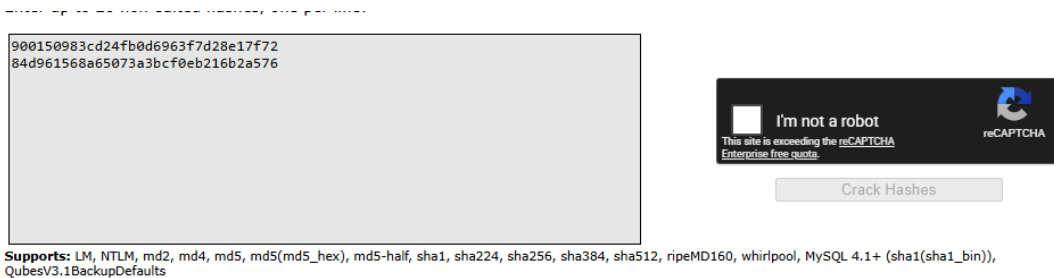


Figurbilde 69: Melding som kommer frem når en setter [ip-adressen]:42420

Dette i seg selv er ikke problematisk da innholdet er bare en tekst med enkel info, men anbefaling om å lukke unødvendige porter er fortsatt gjeldende. Inkludert med det, så bør nettsiden heller ikke bruke HTTP som protokoll, da det er usikkert ettersom all data som blir sendt gjennom HTTP er ukryptert. Det betyr at data blir sendt i plaintext format og da er det lett for angripere å lese innholdet ved å sniffe på koblingen gjennom verktøy som Wireshark

Sårbarhet 10	Kryptering sårbarhet
Medium risiko	Passord hashet med MD5 hashing algoritmen

Etter at en har klart å innhente kredensialene til brukere gjennom SQL-injection sårbarheten, så var det slik at noen passord brukte MD5 hashing algoritmen, når en knakk dem i “crackstation.net”. MD5 er en utdatert hashing algoritme som er sårbar for “collision” angrep, som er en type angrep der to ulike passord input gir den samme hashverdien. Det gjør at en angriper kan forbikjøre autentifisering, selv om passordene i plaintext er ulike da hashen er lik for begge passord. Inkludert med det, så er det heller ikke lagt til salter på de samme passordene, ettersom crackstation bruker rainbow tables eller prekompileerte lookup tables for å matche passordhashene med hashen i crackstation sin database, som vist i skjermbildet under, hvor crackstation dekrypterte passordhashene for brukerne “abc” og “bengt” til plaintext:



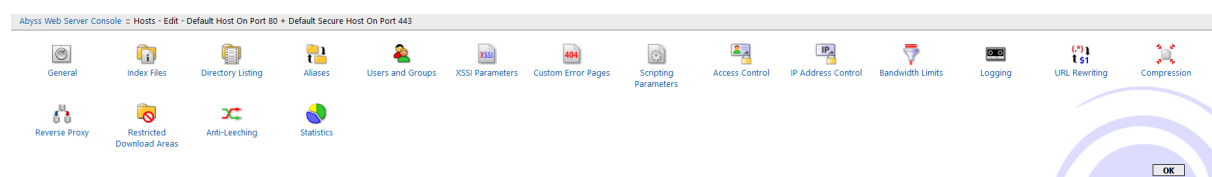
Hash	Type	Result
900150983cd24fb0d6963f7d28e17f72	md5	abc
84d961568a65073a3bcf0eb216b2a576	md5	superman

Figurbilde 70: Passordhashene til “bengt” og “abc”, dekryptert til plaintext i crackstation

Anbefaling er å unngå å bruke MD5 algoritmen for å hashe passord, samtidig som en implementerer salter på passordene. Dette er allerede gjort for majoriteten av passordene, men som vist i sårbarheten med SQL-injection, så trenger angripere bare 1 brukernavn og passord med dårlig hashing algoritme for å kunne dumpe alle kredensialene til brukerne og eventuelt mer fra databasene. Salting gjør at bruken av rainbow tables blir udugelig, inkludert i crackstation, da salting legger til en tilfeldig byte i hashen, noe som gjør at selv to identiske password i plaintext ser ulikt ut når det er gjort om til hashformat. Noen av de andre passordene ser ut til å ha inkludert en salt på seg, noe som er bra, men bør implementeres på alle passord i fremtiden

Sårbarhet 11	OS sårbarhet
Medium risiko	Blind SSI injection

Basert på informasjon som vist i “Informationell 3”, så var det etterlatt kommandoer i kommentarene til “guestbook.php”. Etter litt research på hva kommandoene innebar, så ble det bekreftet at dette var SSI kommandoer og da ønsket en å se om SSI injection var mulig. For at Blind SSI injection skulle fungere, så måtte en gå til abyss web server consolen, trykke på “Configure” i hosts boksen, vist i figurbilde 67 og trykke på “XSSI Parameters” valget:



Figurbilde 71: “XSSI parameters” valget når en trykker på configure

Da møter en på valgene i “XSSI Parameters” siden, der en måtte legge til php som en del av extensions og krysse for “Process “#Exec cmd” Directive”:

Enable XSSI Processing ?

XSSI Error Message ? :

Time Format String ? :

Abbreviated File Size ?

Process "#Exec cmd" Directives ?

Associated Extensions ? :

Extension		
shtml		
shtm		
stm		
html		
php		

Add

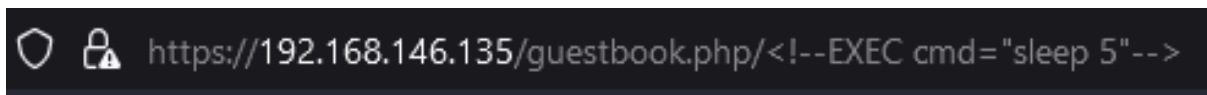
#Exec cgi" Search Paths ? :

Virtual Path		
Empty		

Add

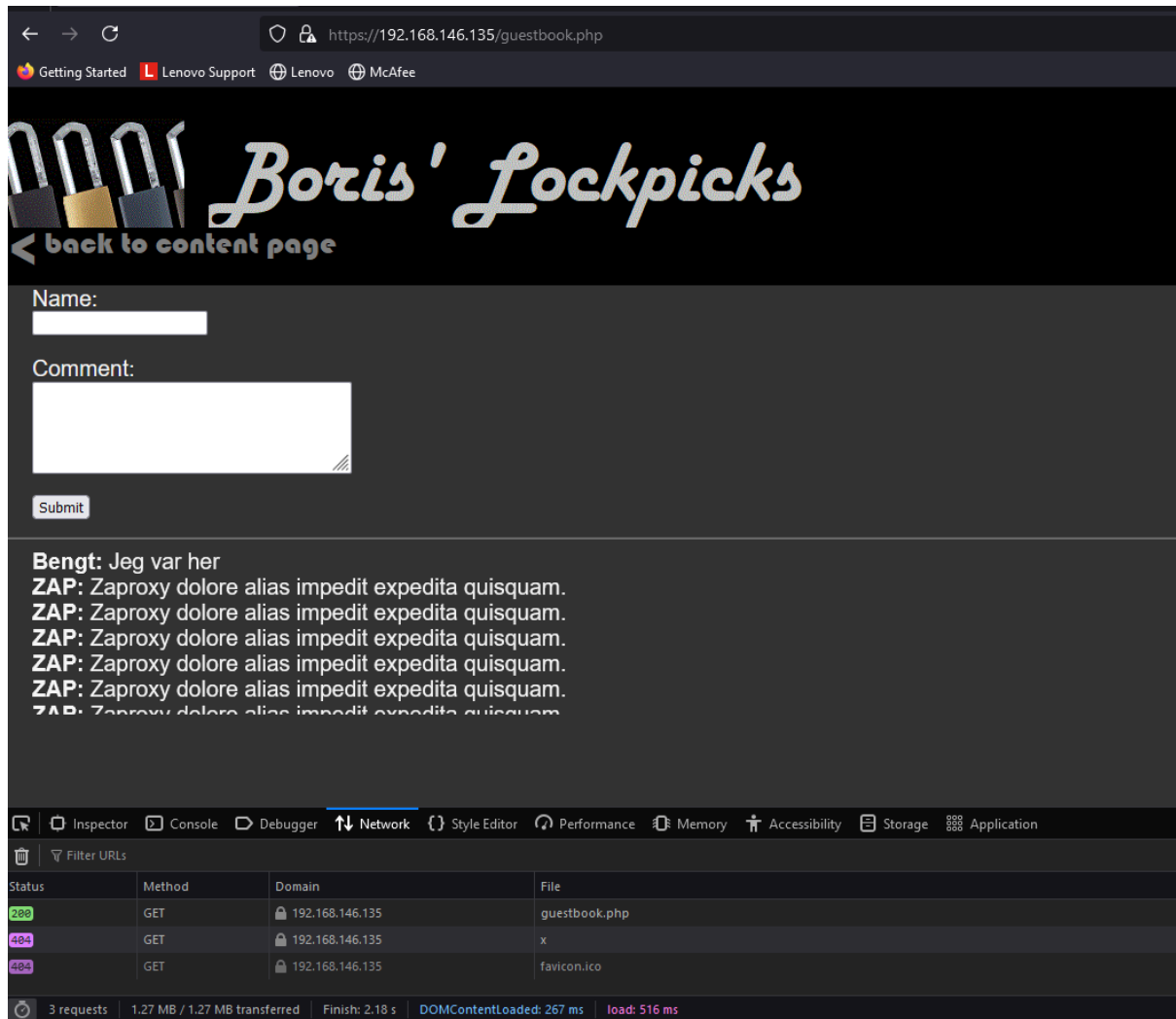
Figurbilde 72: Avkryssing av “Process “#Exec cmd“ Directives” og inkludering av php extension

Etter å ha sjekket for “Process “#Exec cmd” Directives” og lagt til php som extension, så testet en disse kommandoene i siden ved å legge de bak “guestbook.php/”, som vist i skjermbildet under:



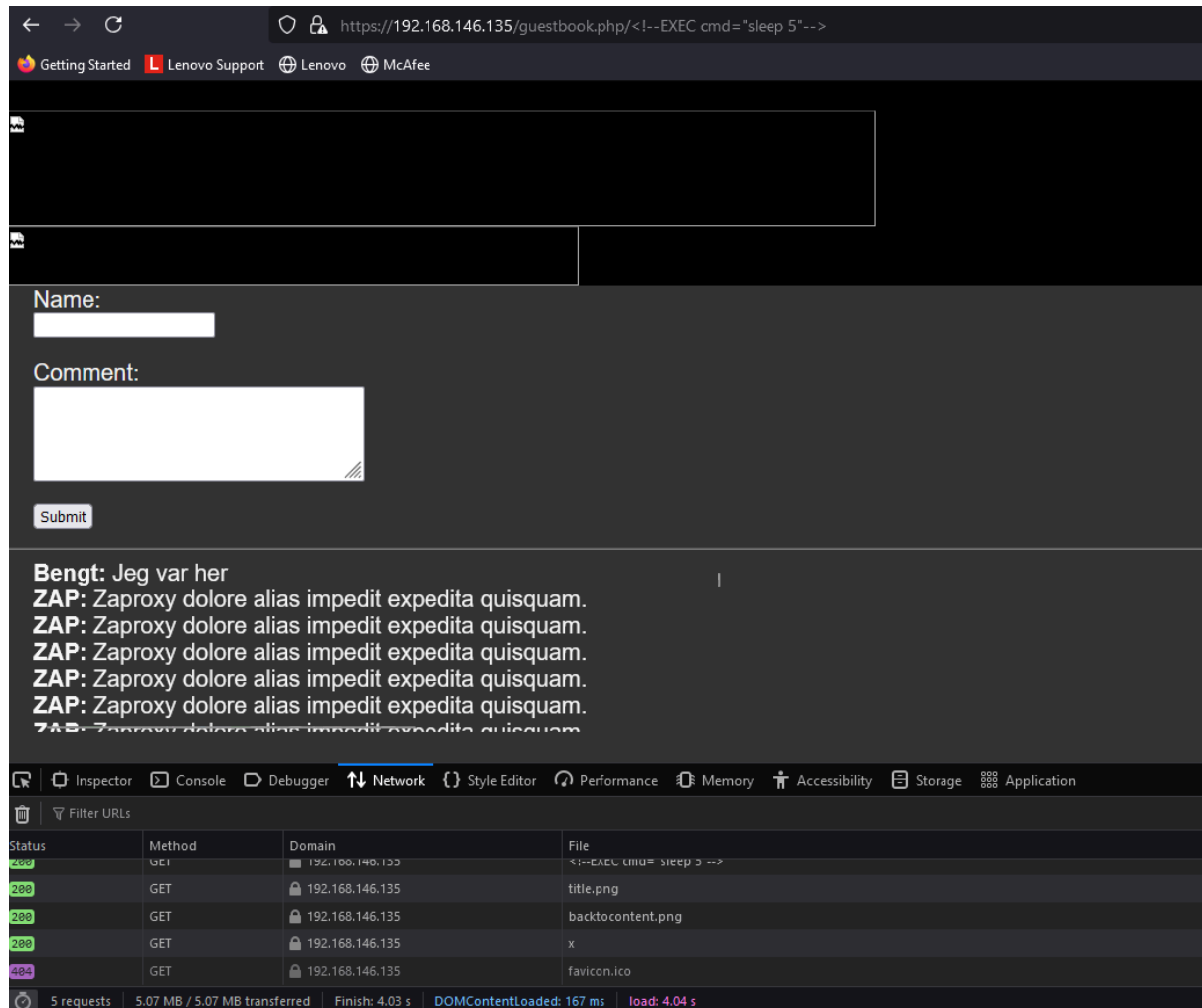
Figurbilde 73: Testing av SSI kommando i URL-en

Under normale tilstander, når brukerne laster ned nettsiden i "guestbook.php", så tar det en periode rundt over 500ms og under 2.5 sekunder å laste ned siden:



Figurbilde 74: Tid siden tar for å laste ned webpagen etter en refresh, før bruk av Exec kommando

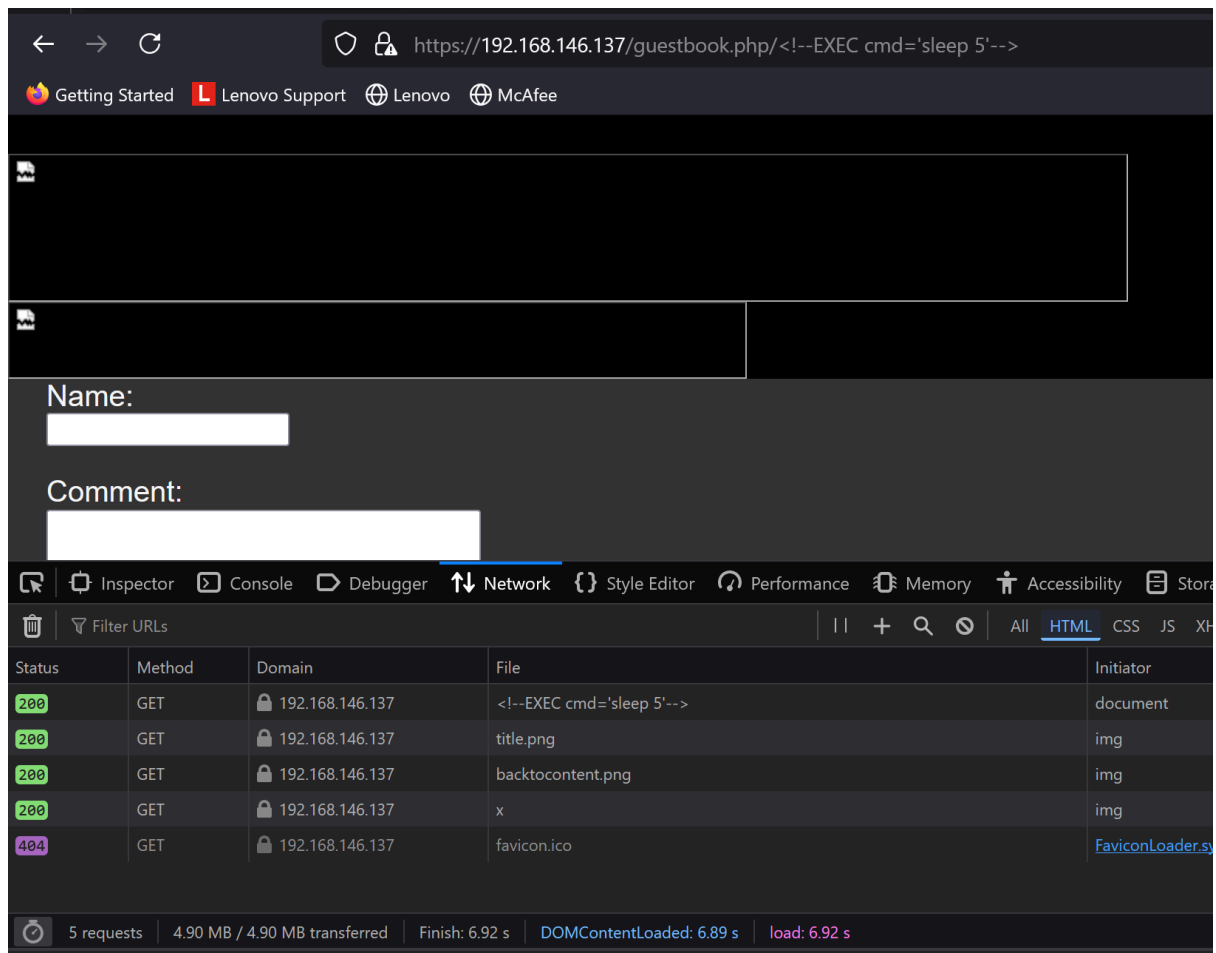
Når en legger til kommandoen “<!--EXEC cmd="sleep 5"-->” i URL-en, så tar nettsiden mellom 2.5 og 4 sekunder for å laste ned:



Figurbilde 75: Etter kjøringen av kommandoen så økte load tiden til 4 sekunder for siden

Tidene for begge scenarioene varierer som sagt, men i scenario to er den laveste tiden en klarer å få på 2.3 sekunder. Ulike tider handler mer om nettverk og server response, ettersom hver refresh gir forskjellige tider for begge scenarior og ikke at angrepet fungerer eller ei.

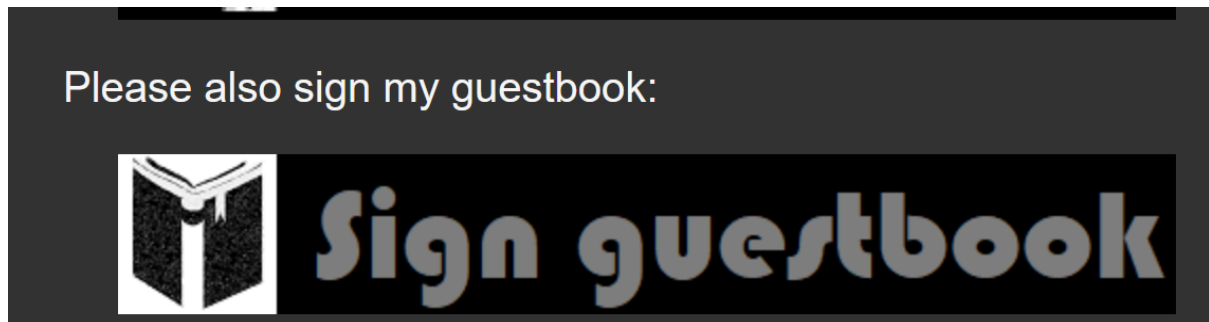
Utførte angrepet på nytt etter en stund senere og har nå bekreftet at blind SSI injection er mulig, ettersom nettleseren bruker nesten 7 sekunder for å laste ned siden fullstending.



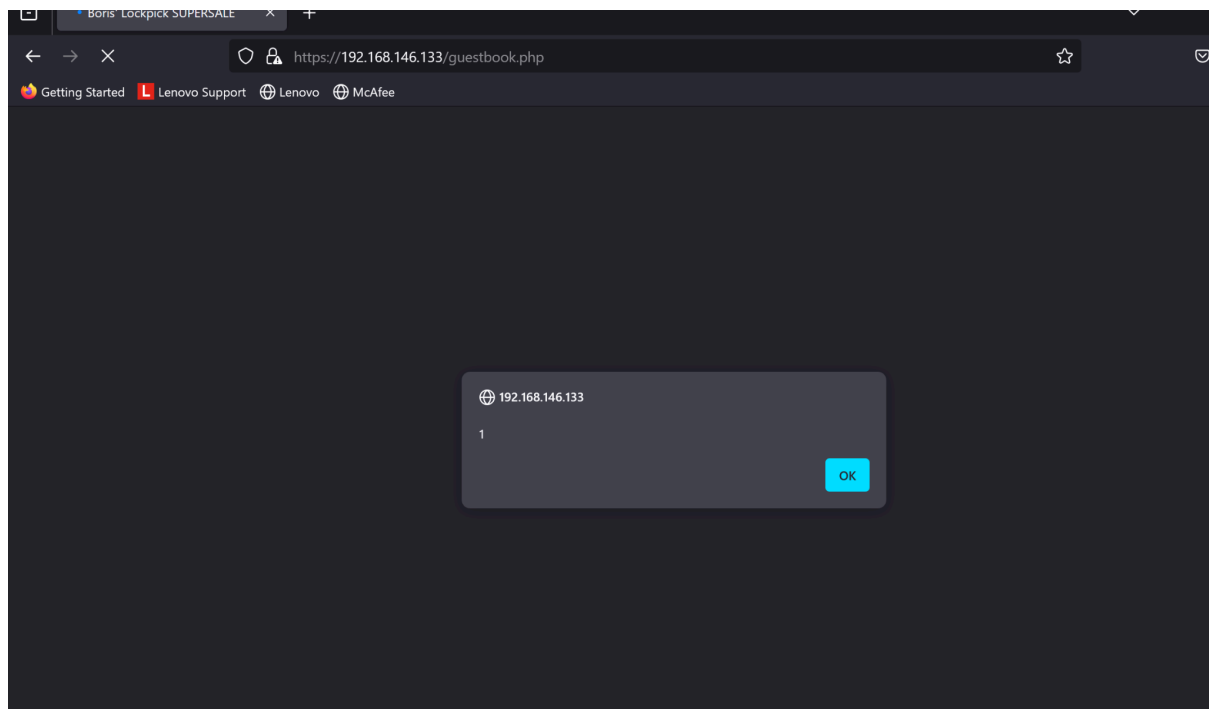
Lav risiko:

Sårbarhet 12	Webbasert sårbarhet
Lav risiko	Stored XSS

Når brukeren trykker på knappen “Sign Guestbook”, møter brukerne på en alert som vist i skjerm bilde 78. Denne alerten gjentar seg flere ganger inntil brukerne sjekker av for å ignorere den gjentakende alert boksen:



Figurbilde 77: “Sign guestbook” knappen i nettsiden



Figurbilde 78: Alerten “1” kommer opp flere ganger når man trykker på “Sign guestbook” knappen

Stored XSS kan medføre konsekvenser hvor en angriper kan lagre en script, der scriptet aktiveres når offeret går til siden og kompromitterer offeret. I motsetning til Reflected XSS, så trenger ikke angrepet at angriperen utfører spesifikke handlinger for å aktiverer den, der de kan legge til exploiten i siden og vente på at brukere kommer borti den.

Anbefaling er å implementere “Output encoding” i Javascript koden. “Output encoding” er innebygget i HTML og Javascript, der oppsettet er forskjellige. Uavhengig av oppsett, så innebærer bruken av “Output encoding” at en kan vise frem data som brukeren har skrevet inn i på en trygg måte, da det forhindrer at variabler blir lest som kode i stedet for tekst i nettsiden. Sanitering av input-feltene er også et ekstra lag med beskyttelse, slik at man forhindrer at brukerne kan legge til <script> tags i feltene. CSP bør også bli implementert, da det gir ekstra beskyttelse mot stored XSS, hvis den er konfigurert til å forhindre at spesifikke sider blir lastet opp gjennom allow lister.

Sårbarhet 13	Mangler av sikkerhetsmekanismer
Lav risiko	Cookie uten samesite attribute

Alle cookies som er synlig i siden har “SameSite Attributet” til “None”:

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
borsilp_basket	9117	192.168.146.140	/	Session	18	false	false	None	Sun, 17 May 2026 09:30:49 GMT
logon_session	1378	192.168.146.140	/	Sun, 24 May 2026 09:30:35 GMT	17	false	false	None	Sun, 17 May 2026 09:30:35 GMT

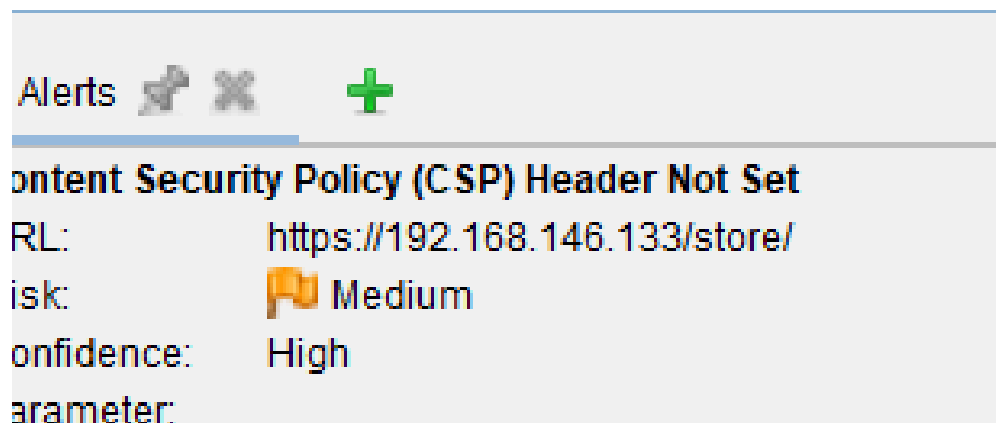
Figurbilde 79: Både “borsilp_basket” og “logon_session” har “SameSite” til none

Dette gjør det mulig for angriperen å utføre clickjacking angrep mot siden ved å modifisere knappene som er fremvist i sårbarhet nummer 5. Anbefaling er å sette alle cookies til å ha “SameSite Attributet” til å være enten “lax” eller “strict”, der “strict” er den mest ideelle.

Sårbarhet 14	Mangler av sikkerhetsmekanismer
Lav risiko	Mangel av CSP headers

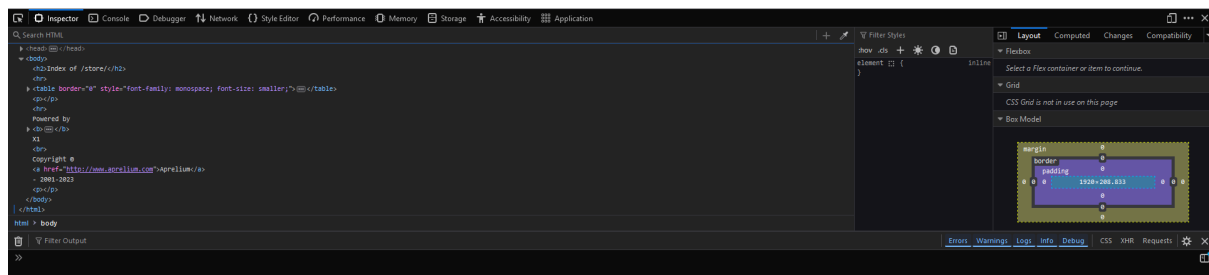
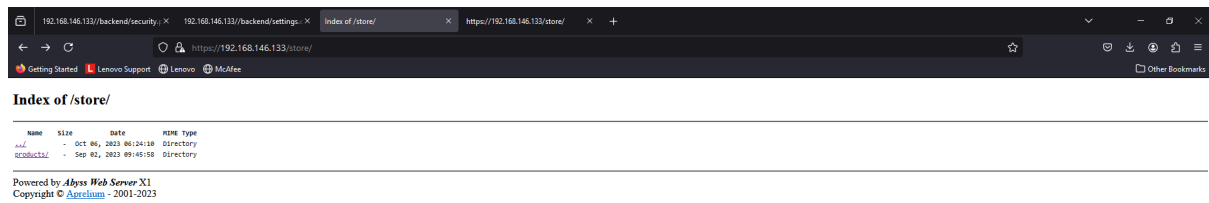
Inkludert med at nettsiden mangler cookies med “SameSite attribute” så mangler siden også CSP headers. Dette er HTML tagger som bidrar sammen med “SameSite attributter” å forhindre Clickjacking angrep mot siden, da mangel av CSP headers gjør det lettere for angriperen å utføre clickjacking angrep, som vist i figurbilde 52-55. Anbefaling er å heller bruke CSP-ancestor-frames, da det bidrar til bedre sikkerhet mot clickjacking angrep, men dersom nettlesere er en eldre versjon og som ikke kan bruke CSP-ancestor-frames, så bør man implementere CSP headers.

I OWASP ZAP:



Figurbilde 80: Informasjon om hvor sårbarheten ligger i nettsiden

Siden som mangler CSP headers:

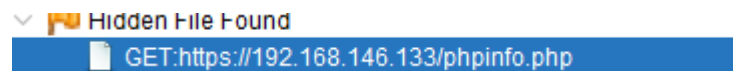


Figurbilde 81: Demonstrasjon av siden som faktisk mangler CSP headers

Informasjon:

Informasjonell 1	Ekspontert info
Informasjon	PHP versjonsnummer info

OWASP ZAP oppdaget en skjult side på nettsiden. Da en skrev det i url-en så kommer PHP versjonsnummer info, som vist is figurbilde 83:



Figurbilde 82: “phpinfo.php” var skannet av OWASP ZAP

PHP Version 7.3.31-1~deb10u5	
System	Linux osboxes 4.19.0-5-amd64 #1 SMP Debian 4.19.37-5 (2019-06-19) x86_64
Build Date	Sep 4 2023 21:49:25
Server API	CGIFastCGI
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php/7.3/cgi
Loaded Configuration File	/etc/php/7.3/cgi/php.ini
Scan this dir for additional .ini files	/etc/php/7.3/cgi/conf.d
Additional .ini files parsed	/etc/php/7.3/cgi/conf.d/10-mysqlnd.ini, /etc/php/7.3/cgi/conf.d/10-opcache.ini, /etc/php/7.3/cgi/conf.d/10-pdo.ini, /etc/php/7.3/cgi/conf.d/20-calendar.ini, /etc/php/7.3/cgi/conf.d/20-ctype.ini, /etc/php/7.3/cgi/conf.d/20-exif.ini, /etc/php/7.3/cgi/conf.d/20-fileinfo.ini, /etc/php/7.3/cgi/conf.d/20-ftp.ini, /etc/php/7.3/cgi/conf.d/20-gettext.ini, /etc/php/7.3/cgi/conf.d/20-iconv.ini, /etc/php/7.3/cgi/conf.d/20-json.ini, /etc/php/7.3/cgi/conf.d/20-mysqli.ini, /etc/php/7.3/cgi/conf.d/20-pdo_mysql.ini, /etc/php/7.3/cgi/conf.d/20-phar.ini, /etc/php/7.3/cgi/conf.d/20-posix.ini, /etc/php/7.3/cgi/conf.d/20-readline.ini, /etc/php/7.3/cgi/conf.d/20-shmop.ini, /etc/php/7.3/cgi/conf.d/20-sockets.ini, /etc/php/7.3/cgi/conf.d/20-sysmsg.ini, /etc/php/7.3/cgi/conf.d/20-sysvsem.ini, /etc/php/7.3/cgi/conf.d/20-sysvshm.ini, /etc/php/7.3/cgi/conf.d/20-tokenizer.ini
PHP API	20180731
PHP Extension	20180731
Zend Extension	320180731
Zend Extension Build	API320180731.NTS
PHP Extension Build	API20180731.NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	disabled
IPv6 Support	enabled
DTrace Support	available, disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk, convert.iconv.*

This program makes use of the Zend Scripting Language Engine:
 Zend Engine v3.3.31, Copyright (c) 1998-2018 Zend Technologies
 with Zend OPcache v7.3.31-1~deb10u5, Copyright (c) 1999-2018, by Zend Technologies

Figurbilde 83: Informasjon om PHP versjonsnummer

Her finner man også masse andre ting som for eksempler at FTP protokollen er aktiv og at nettsiden har en aktiv SSL sertifikat:

ftp	
FTP support	enabled
FTPS support	enabled

Figurbilde 84: Support for FTP og FTPS er aktiv

openssl

OpenSSL support	enabled
OpenSSL Library Version	OpenSSL 1.1.1c 28 May 2019
OpenSSL Header Version	OpenSSL 1.1.1n 15 Mar 2022
Openssl default config	/usr/lib/ssl/openssl.cnf

Figurbilde 85: OpenSSL support er aktiv. Versjonnummer for OpenSSL header og library er synlig. Default config pathen er også synlig

Bunnen av siden inneholder informasjon av konfigurasjon av nettsiden, kalt for PHP variables:

PHP Variables

Variable	Value
\$_COOKIE[logon_session]	439
\$_SERVER[JOURNAL_STREAM]	9:20361
\$_SERVER[PATH]	/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
\$_SERVER[INVOCATION_ID]	43f19a8743a844829ea199070e4c88bd
\$_SERVER[LANG]	en_US.UTF-8
\$_SERVER[PWD]	/
\$_SERVER[WSREP_START_POSITION]	no value
\$_SERVER[FCGI_X_PIPE_]	/tmp/abws-fcgi-00000001-0000033700000000-00000006
\$_SERVER[X_ABYSS_STATS_SERVER_COMPRESS_ORIGINAL_OUTPUT]	34410959
\$_SERVER[X_ABYSS_STATS_SERVER_COMPRESS_OUTPUT]	5054526
\$_SERVER[X_ABYSS_STATS_SERVER_COMPRESS_TOTAL]	3913
\$_SERVER[X_ABYSS_STATS_HOST_COMPRESS_ORIGINAL_OUTPUT]	34403087
\$_SERVER[X_ABYSS_STATS_HOST_COMPRESS_OUTPUT]	5051782
\$_SERVER[X_ABYSS_STATS_HOST_COMPRESS_TOTAL]	3909
\$_SERVER[X_ABYSS_STATS_HOST_CURRENT_UPTIME]	751
\$_SERVER[X_ABYSS_STATS_HOST_UPTIME]	684711
\$_SERVER[X_ABYSS_STATS_HOST_OUTPUT]	14473497931
\$_SERVER[X_ABYSS_STATS_HOST_NOTMODIFIED]	83
\$_SERVER[X_ABYSS_STATS_HOST_IMAGE]	27677
\$_SERVER[X_ABYSS_STATS_HOST_HTML]	280395
\$_SERVER[X_ABYSS_STATS_HOST_ERROR]	228135
\$_SERVER[X_ABYSS_STATS_HOST_TOTAL]	489891
\$_SERVER[X_ABYSS_STATS_SERVER_CURRENT_UPTIME]	751
\$_SERVER[X_ABYSS_STATS_SERVER_UPTIME]	1444665
\$_SERVER[X_ABYSS_STATS_SERVER_OUTPUT]	14473723444
\$_SERVER[X_ABYSS_STATS_SERVER_NOTMODIFIED]	93
\$_SERVER[X_ABYSS_STATS_SERVER_IMAGE]	27685
\$_SERVER[X_ABYSS_STATS_SERVER_HTML]	280434

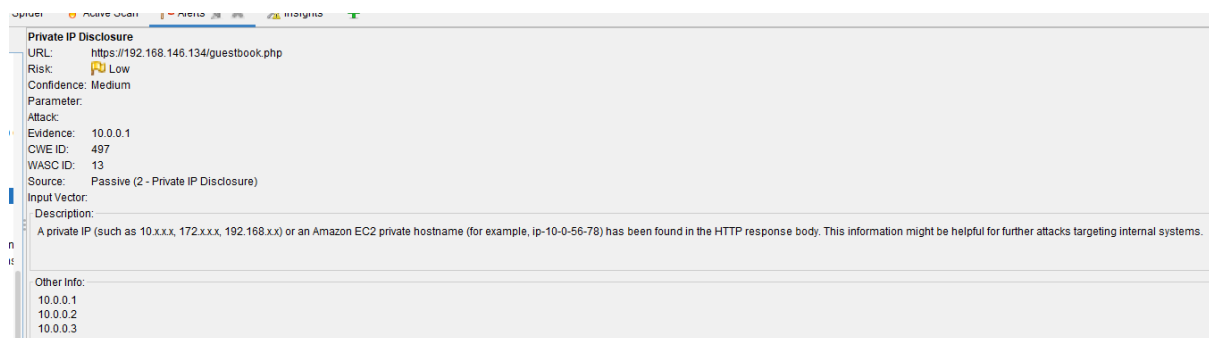
Figurbilde 86: PHP variabler seksjonen av siden

Denne siden alene utgjør ikke noe fare for bedriften, men den kan hjelpe angripere å kartlegge servicene og service versjonsnummere i applikasjoner, hashing funksjoner som er implementert og andre skjulte filstier i nettsiden som OWASP ZAP ikke har klart å skanne. Anbefalingene er å forhindre at konfigurasjonssider er lett tilgjengelig på nett og for

scannerverktøy å ta opp, slik at kartlegging av siden blir vanskelige for angripere og slik at de ikke kan tukle med konfigurasjoner og ødelegge nettsiden.

Informasjonell 2	Ekspontert info
Informasjon	Ekspontert private IP-adresser

OWASP ZAP oppdaget også private IP-adresser i kommentarfeltet i “guestbook.php”. Da dette ikke direkte leder til en sårbarhet til siden, vil det derimot gjøre at angriperen får en innsikt i bedriftens infrastruktur og kan potensielt hjelpe angriperen med å angripe mot bedriftens interne systemer:



Figurbilde 87: Private IP-adresser oppdaget i OWASP ZAP

```
from dual): Zaproxy dolore alias impedit expedita quisquam.  
ZAP / (SELECT UTL_INADDR.get_host_name('10.0.0.1')  
from dual union SELECT  
UTL_INADDR.get_host_name('10.0.0.2') from dual  
union SELECT UTL_INADDR.get_host_name('10.0.0.3')
```

Figurbilde 88: De private IP-adressen “10.0.0.1”, “10.0.0.2”, og “10.0.0.3” synlige i kommentarfeltet i “guestbook.php”

Anbefaling er å unngå å vise frem private IP-adresser ved å konfigurere web serveren til å filtrere private IP-adresser. Andre anbefalinger er å unngå at adressene kommer frem i banners eller i error meldinger og implementering av identifiers slik at angriperne ikke klarer å danne en kobling mellom identifiseren og IP-adressen.

Informasjonell 3	Ekspontert info
Informasjon	Uttkommentert sql og commandline kommandoer i kildekoden

I "guestbook.php", så klarte en å finne command line executions og sql spørringer utkommentert i siden. Dette er mest sannsynlige kommentarer som ikke ble slettet vekk under produksjon av utviklerne og som nå ligger ute i prototype versjon av nettsiden:

```
<br> [overflow]
<b>ZAP:</b> [overflow]
<!--#EXEC cmd="ls /"-->
<br> [overflow]
<b>ZAP:</b> [overflow]
">
<!--#EXEC cmd="ls /"-->
<
<br> [overflow]
<b>ZAP:</b> [overflow]
<!--#EXEC cmd="dir \"-->
<br> [overflow]
<b>ZAP:</b> [overflow]
">
<!--#EXEC cmd="dir \"-->
<
```

Figurbilde 89: Uttkommenterte SSI kommandoer i kildekoden

```
<br> [overflow]
; select "java.lang.Thread.sleep"(15000) from INFORMATION_SCHEMA.SYSTEM_COLUMNS where TABLE_NAME = 'SYSTEM_COLUMNS' and COLUMN_NAME = 'TABLE_NAME' -- :
```

Figurbilde 90: SQL kommandoer i kildekoden

Da disse kommandoene ikke utgir en sårbarhet i seg selv, så hjelper det fortsatt angripere å kartlegge kommandoer som er mulig å utføre på siden og SQL informasjon som tilhører backenden. Denne informasjonen kan hjelpe angripere utføre command execution på nettsiden, dersom de finner en page som tillater det og dersom siden ikke saniterer for spesialtegn, så er det mulig for angriperen å utføre SQL-injection angrep, enten manuelt eller via SQLmap. Anbefaling er å fjerne vekk alle kommentarer på sidene før en publiserer koden til nettet.

Informasjonell 4	Ekspontert info
Informasjon	Filstier til filer i nettsiden inni html tags

Etter grundig gjennomgang av innholdet i kommentarfeltet i “guestbook.php” siden, ble det oppdaget relative filstier som “c:\Windows\system.ini”, “\WEB-INF\web.xml” og “/etc/passwd/”, inkludert absolutte filstier for “Windows\system.ini” og “/etc/passwd”:

```

\WEB-INF\web.xml
<br> overflow
<b>ZAP:</b> overflow
thisshouldnotexistandhopefullyitwillnot

```

Figurbilde 91: Filstien for “web.xml” og teksten “thisshouldnotexistandhopefullyitwillnot”

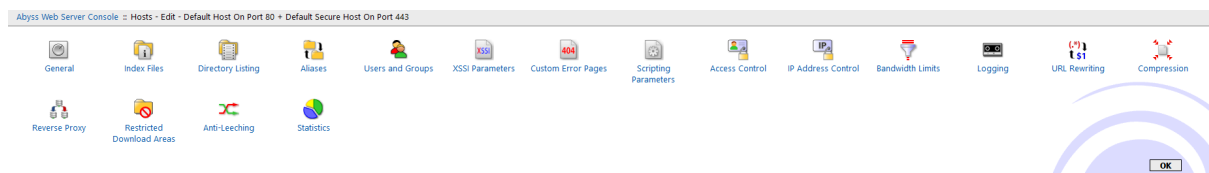
```

<b>ZAP:</b> overflow
c:\Windows\system.ini
<br> overflow
<b>ZAP:</b> overflow
../../../../../../../../../../../../../../../../Windows\system.ini
<br> overflow
<b>ZAP:</b> overflow
/etc/passwd
<br> overflow
<b>ZAP:</b> overflow
../../../../../../../../../../../../../../../../etc/passwd

```

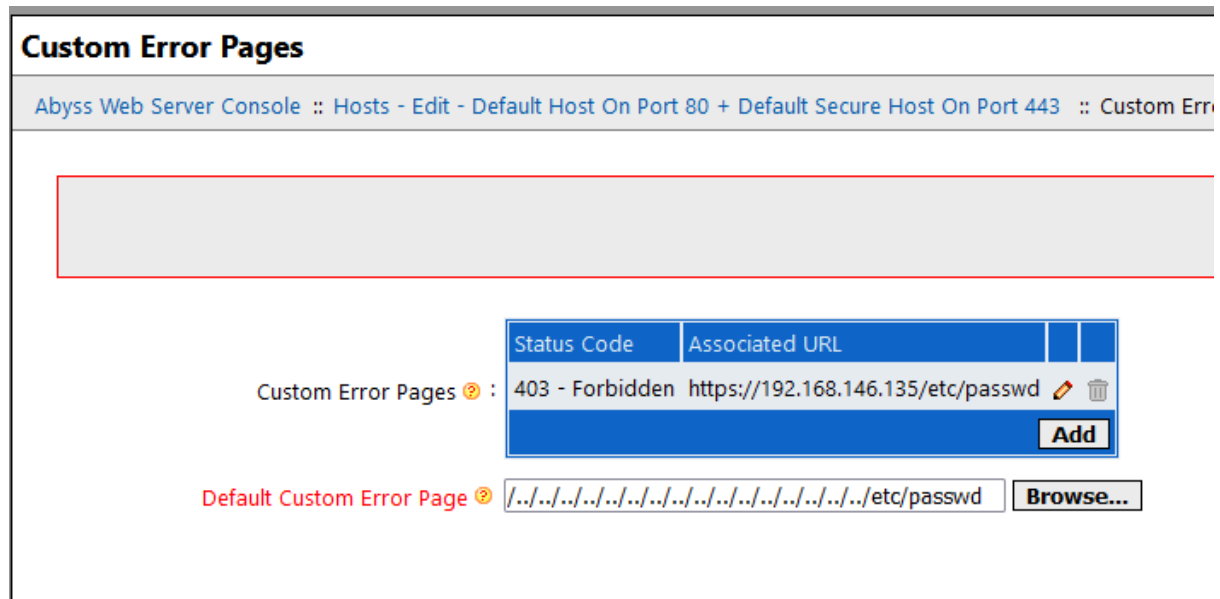
Figurbilde 92: Relative og absolutte filstier for “system.ini” og “passwd”

Ut av nysgjerrighet testet en ut om en kunne få en respons når en la til den absolutte filstien til “/etc/passwd” som en error page i nettsiden. I abyss web server console, trakk en på “Custom Error pages”:



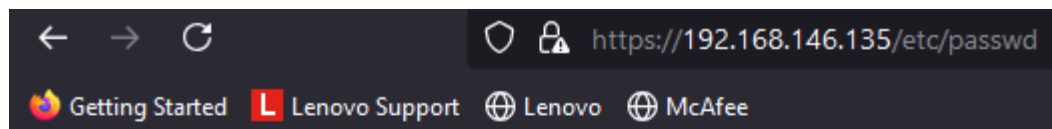
Figurbilde 93: “Custom Error pages” valget når en trykker på configure

La til absolute filstiene i "Custom Error Pages" og "Default Custom Error Page", der en valgte error code 403 for "Custom Error Pages":



Figurbilde 94: Input av relative og absolutt filsti for "/etc/passwd"

For at en skulle få disse sidene opp, så er en nødt til å skrive en invalid URL i søket. Resultatene ble ikke som en hadde ønsket:



Error 404

Not Found

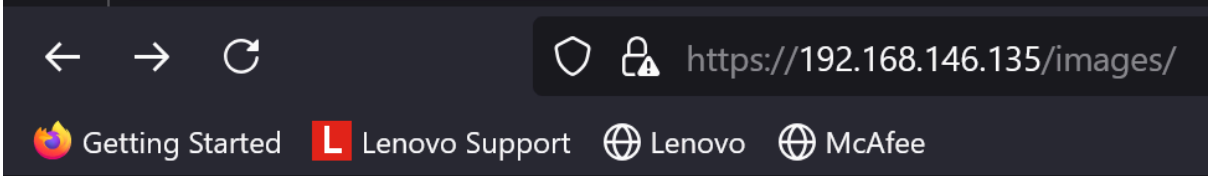
Powered by *Abyss Web Server X1*
Copyright © [Aprelium](#) - 2001-2023

Figurbilde 95: En får opp "Error 404" i stedet for 403

Da disse ikke utgjøre en risiko i seg selv, så er anbefaling fortsatt å ikke legge til filstier, selv om disse ikke gjør noe i denne situasjonen. Andre tiltak er å validere brukerinput før nettsiden prosesserer den, slik at den inputta filstien er valid og ikke inneholder tegn som forårsaker at skjulte filer blir fremvist. Andre tiltak er å standardisere filstien, samtidig som den blir plassert i hoved directorien (abysswsd) i denne situasjonen

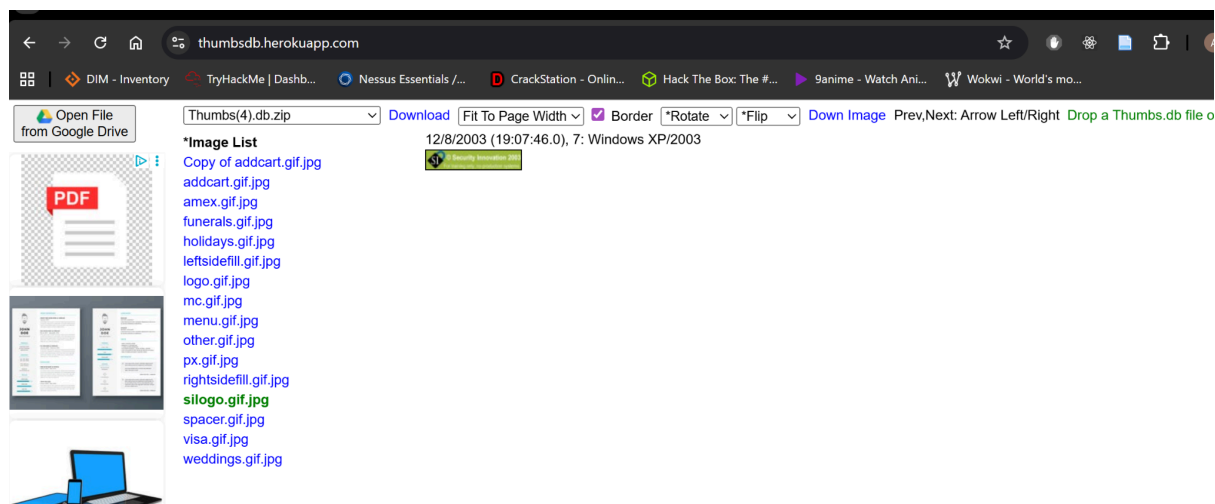
Informasjonell 5	Ekspontert info
Informasjon	OS informasjon synlig fra png artifakter

I index av images, så var det funnet en fil kalt "Thumbs.db" som en kunne laste ned på maskinen, som vist i figurbilde 96. En dro til nettsiden "thumbsdb.herokuapp.com" for å åpne innholdet til [thumbs.db](https://thumbsdb.herokuapp.com), der det var .jpg bilder fra Windows XP/2003 operativsystemet:



Name	Size	Date	MIME Type
../	-	Oct 06, 2023 06:24:10	Directory
Thumbs.db	31.50 KB	Feb 06, 2017 02:37:38	application/octet-stream
addtobasket.png	2.08 KB	Feb 26, 2023 17:45:00	image/png

Figurbilde 96: "[Thumbs.db](#)" og "[addtobasket.png](#)" i indeks av "images"



Figurbilde 97: Innholdet i filen "Thumbs(4).db"

Da bildene er fra Windows XP/2003 maskiner, lurte en på om en kunne bruke en exploit som eternalblue for bevege seg videre til dypere tilgang. Da kjørte en nmap script scan for å se om det var situasjonen:

```
(kali@kali)~$ nmap -sV --script smb-vuln-ms17-010 192.168.146.140
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-25 05:22 EDT
Nmap scan report for 192.168.146.140
Host is up (0.0029s latency).
Not shown: 988 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
9/tcp     open  discard?
13/tcp    open  daytime
21/tcp    open  ftp?
22/tcp    open  ssh          OpenSSH 7.9p1 Debian 10+deb10u3 (protocol 2.0)
37/tcp    open  time         (64 bits)
|_rfc868-time: 2026-05-25T09:25:17
53/tcp    open  domain       ISC BIND 9.11.5-P4-5.1+deb10u9 (Debian Linux)
79/tcp    open  finger?
80/tcp    open  http         Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
|_http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
443/tcp   open  ssl/http     Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
|_http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
9999/tcp  open  http         Abyss httpd 2.16.9.1-X1 (AbyssLib/2.16.9.1)
|_http-server-header: Abyss/2.16.9.1-X1-Linux AbyssLib/2.16.9.1
MAC Address: 00:0C:29:08:07:C1 (VMware)
Service Info: Host: OSBOXES; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

Figurbilde 98: Nmap scannen hvor en får opp alle åpen porter med bekreftelse på at VM bruker Linux i stedet for Windows

Dette bekrefter at VM-en ikke kjører Windows, noe som gjør at eternalblue payloaden ikke kan bli brukt for å få kontroll over maskinen. Konsekvensen i denne situasjonen blir ikke drastisk, da root access ikke er oppnådd, men det anbefales at slike artifakter slettes vekk fra systemet.

Metodikk:

En har hovedsakelig brukt funnene fra "Reconnains fasen" og utført manuelt testing på de forskjellige portene og sidene i nettside. Boken "Kali Linux 2018: Assuring Security by Penetration Testing" og "OWASP Testing guide" for supplerende testing mot andre, mindre kjente angrepsvektorer.

Alle anbefalinger og tiltak for hvert enkelte funn er hentet fra owasp.org sin dokumentasjon. Testen følger et whitebox format, med ekstern tillatelse. TLS/SSL er self-signed ifølge eieren, så det telles ikke som en sårbarhet i dette penetrasjonsoppdraget

Liste over alle verktøy:

OWASP ZAP = Kartlegging av sårbarheter i VM-en, inkludert gjennom spiders

NMAP = Brukt for å skanne etter åpne porter og tilkoblede servicer i VM-en

NESSUS = Skanning av potensielle sårbarheter til VM-en gjennom bruken av nettsida

Wireshark = Sniffing av nettverk for å fange IP-adressen for testing

Firefox = Browser for å hoste VM-en sin nettside

Visual Studio Code = IDE for å utføre statisk kodeanalyse